Software Requirements Specification
Software Engineering

Team #14, Troitsky Toolbox
Pedro Azevedo
Mason Azzopardi
Michael Roberts
Jack Easton

Table 1: Revision History

Date	**Developer(s)**	**Change**
10/02/2026	Pedro	SRS template (Meyer) adopted; section files created.
10/07/2026	Pedro	G.1 drafted from problem statement.
10/08/2026	Jack	P.6 risks and mitigation analysis added.
10/11/2026	Jack	P.6 expanded.
10/13/2026	Pedro	Formatting aligned with other team documents.

This document follows the requirements documentation structure presented in the Handbook of requirements and business analysis, by Bertrand Meyer [**?**]. Changes made to the template are listed in Table 21 (Appendix A.3).

Contents

E.1 Glossary . . . 5
E.1.1 Terms . . . 5
E.1.2 Acronyms . . . 6

G Goals **7**
G.1 Context and overall objective . . . 7
G.2 Current situation . . . 7
G.3 Expected benefits . . . 7
G.4 Functionality Overview . . . 9
G.5 High-level usage scenarios . . . 10
G.6 Limitations and exclusions . . . 10
G.7 - Stakeholder and Requirements Sources . . . 11

E Environment **13**
E.2 Components . . . 13
E.3 Constraints . . . 14
E.4 Assumptions . . . 14
E.5 Effects . . . 15
E.6 Invariants . . . 16

S System **16**
S.1 Components . . . 16
S.2 Functionality . . . 22
S.2.1 Web Server . . . 22
S.2.2 Access Control System . . . 23
S.2.3 Toolbox Broker . . . 24
S.2.4 Knowledge Repository . . . 25
S.2.5 Model Analysis System . . . 26
S.2.6 Material Testing Manager . . . 30
S.2.7 Knowledge Transfer Manager . . . 32
S.2.8 System-Wide Requirements . . . 33
S.3 Interfaces . . . 33
S.4 Detailed usage scenarios . . . 34
S.5 Prioritization . . . 40
S.5.1 Web Server . . . 40
S.5.2 Access Control System . . . 41
S.5.3 Toolbox Broker . . . 42
S.5.4 Knowledge Repository . . . 43
S.5.5 Model Analysis System . . . 44
S.5.6 Material Testing Manager . . . 47
S.5.7 Knowledge Transfer Manager . . . 48
S.5.8 System-Wide Requirements . . . 49
S.5.9 Usage Scenarios . . . 49
S.5.10 Interfaces . . . 50

S.5.11 Safety and Security Requirements . . . 51
S.5.12 Justification for Prioritization . . . 51
S.5.13 Role of Security Concerns . . . 52
S.5.14 Phase-In Plan . . . 52
S.6 Verification and acceptance criteria . . . 54
S.7 Formalization . . . 56
S.7 Formalizations . . . 58

P Project **59**
P.1 Roles and personnel . . . 59
P.1.1 Project Supervisor . . . 59
P.1.2 Toritsky Club Communication Associate . . . 59
P.1.3 Front-end / UI Developer . . . 59
P.1.4 Back-end / Data Developer . . . 59
P.1.5 Quality Assurance / Tester . . . 59
P.1.6 Supervisor . . . 60
P.1.7 Teaching Assistant . . . 60
P.2 Imposed technical choices . . . 60
P.3 Schedule and milestones . . . 60
P.4 Tasks and deliverables . . . 61
P.5 Required technology elements . . . 63
P.6 Risks and mitigation analysis . . . 64
P.6.1 Risk Analysis . . . 64
P.6.2 Threat Analysis . . . 65
P.7 Requirements process and report . . . 66
P.7.1 Requirements Survey Report . . . 66
P.7.2 Ongoing and Future Processes . . . 69

References **70**

A Appendix **70**
A.1 Symbolic Constants . . . 70
A.2 Traceability Matrix . . . 75
A.3 Changes from Template . . . 78
A.4 Reflection . . . 79
A.4.1 Team . . . 80
A.4.2 Pedro Azevedo . . . 81
A.4.3 Mason Azzopardi . . . 81
A.4.4 Michael Roberts . . . 82
A.4.5 Jack Easton . . . 82
A.4 Generative AI Use Disclosure . . . 83
A.4.1 AI Tools Used . . . 83
A.4.2 How and Where Used . . . 83
A.4.3 Verification of AI-Generated Content . . . 83
A.5 Reflection . . . 84
A.4.1 Team . . . 84

A.4.2 Pedro Azevedo 85
A.4.3 Mason Azzopardi 86
A.4.4 Michael Roberts 86
A.4.5 Jack Easton 86

E.1 Glossary

Terms and acronyms used in this document. Each entry links to the term's first use in the text. Competition terms follow the 2026 rulebook [**?**, Secs. 1, 3, 5.1, 5.2, 10.5, 10.6].

E.1.1 Terms

- **Coach:** An advisor to a team who does not compete. The rulebook allows at most one coach per team, who may not help build the bridge on competition day. Coaches are graduate students, people who graduated in the last two years, or undergraduates who have competed before.
- **Current model:** A bridge model that a team is developing for the upcoming competition. Only the members of that team can access it (S.1, FR-AC-6).
- **Exit document:** A document that an outgoing executive or team lead completes from the template for their position, so that their successor knows what the position involves (FR-KT-1).
- **Failure load:** The largest load a bridge carries before it fails when loaded at mid-span. The rulebook calls it the ultimate load carrying capacity. Also called load capacity.
- **Handoff document:** Any document that passes knowledge on to future members: exit documents, role records, and onboarding material (S.1). Also called a handover, transition, or knowledge transfer document.
- **Load-to-mass ratio:** A bridge's failure load divided by its mass. The rulebook scores it as structural efficiency.
- **Materials testing team:** The club members who test the competition materials, such as popsicle sticks, glue and floss, and record the results (E.2, S.4.6). Also called the materials testing department.
- **Past design:** A bridge model from a previous competition year. Any member can view it, but no one can modify it. At the end of each competition season, current models become past designs (FR-MA-5, FR-MA-6).
- **Role record:** A record for one executive position that describes its responsibilities, key contacts, and yearly timeline. Unlike an exit document, it is kept up to date across years by the current holder of the position (FR-KT-3).
- **Rule set:** The competition rules for one competition year, as stored and checked by the Rule Compliance System (FR-RC-1, FR-RC-2).
- **Software subteam:** A planned subteam of club members, mainly CAS students, who will maintain and extend the Toolbox once the original developers graduate (G.7, E.3).

- **Team captain:** The member of a competing team who is its primary point of communication with the competition's Organizing Committee.
- **Troitsky club:** The McMaster Troitsky club, the student-run organization that registers and supports McMaster's teams in the Troitsky competition (G.1).
- **Troitsky competition:** An annual bridge-building competition held at Concordia University. Teams of six to eight undergraduate students build a bridge from popsicle sticks, toothpicks, white glue and dental floss, which is then loaded at mid-span until it fails.

E.1.2 Acronyms

Acronym	Expansion
CAD	Computer-Aided Design
CAS	Computing and Software, the McMaster department
CS	Computer Science
ISO	International Organization for Standardization
LGPL	GNU Lesser General Public License
LINQ	Language-Integrated Query
MES	McMaster Engineering Society
MIT	MIT License, a permissive open-source license
OWASP	Open Worldwide Application Security Project
RBAC	Role-Based Access Control
STEP	Standard for the Exchange of Product model data (ISO 10303). Also written STP, after the `.stp` file extension
TCF	Team Contribution Factor
UI	User Interface
VnV	Verification and Validation (also written V&V)
VP	Vice-President

G Goals

G.1 Context and overall objective

The McMaster Troitsky club is a student-run organization that registers and supports 4-6 teams of students [**?**] as they take on the task of modelling, analyzing and manufacturing a bridge out of popsicle sticks, toothpicks, white glue and dental floss. While the club has been successful in the past, results often vary in performance both between years and teams, due to the lack of centralized information and technical support for members. As teams progress through several years of competition, undergoing material testing, bridge design and analysis, and even experimenting with more innovative concepts, skill and knowledge within the club often consolidates within more experienced members or single teams, ensuring that when those members graduate, information is lost to the club. This project seeks to create a centralized information repository and technical support system for the McMaster Troitsky club, ensuring information transfer between years and teams, and facilitating support and onboarding for new members.

G.2 Current situation

Currently, teams operate in a highly individualistic manner, with the greater McMaster Troitsky club covering the administrative details regarding competition registration and providing guidance and feedback when possible. Teams store information through Microsoft Teams, Google Drive and emails between members, with many teams resulting in having a single member in charge of different competition components. Currently, the club as a whole does not have a centralized system in place to store past designs, bridge models or technical details, only having more adiminstrative/executive documents stored. Even tutorials that are consistently useful for teams do not have a consistent access point and therefore have become unreliable as proper support for the club. To address these challenges, for the past 2 years the club has been pushing for more integration between teams, generalizing the coaching positions, providing larger-scale support and creating a stronger administrative presence through timelines and events, but still lacks a service to centralize and make this data available.

G.3 Expected benefits

The Toolbox addresses the problem described in G.1 by replacing the club's informal, individual team-based data storage solutions with consistent services that retain data between competition years. The member survey, described in P.7, outlines the importance of services and tools, providing a strong evidence for core user concerns and the benefits that can be gained from addressing them. The expected benefits of the core user base groups are listed below.

General Members and Teams

- **Self-directed learning for new members:** New members can explore past bridge designs, material test results, and design documentation on their own, rather than relying on experienced members being available to explain them. This lowers the barrier to joining the club and allows new members to contribute to their team sooner.
- **Learning from past designs:** Teams can load past bridge designs into the model viewer and inspect them alongside their recorded results, so that each year's designs build on what previous teams learned instead of starting from scratch.
- **Accessible bridge analysis:** Teams can upload their bridge model and receive an estimate of its failure load and load-to-mass ratio, without needing formal structural modelling knowledge. This allows every member of a team, not only those with analysis experience, to take part in evaluating and improving a design.
- **Early rule compliance checking:** Teams can check their model against the current competition rules before building it, catching violations while they are still easy to fix, rather than after time and materials have been spent on construction.
- **Data-driven material decisions:** Material testing results are recorded in a standard format and compared against previous years, so teams can base their design decisions on the club's accumulated test data. Performance estimates also use material properties measured by the club itself.

Club Executives

- **Reduced support burden:** Much of the technical support and information sharing that currently falls on executives is replaced by self-service access to the Toolbox, freeing executives to focus on running the club.
- **Reliable leadership transitions:** Exit documents and role records give each incoming executive a written account of their position's responsibilities, contacts, and yearly timeline, so that the club's organization does not depend on informal handovers.
- **Consistency across teams:** Because all teams draw on the same repository, test data, and tools, executives can expect a more consistent baseline of knowledge across the four to six teams the club supports each year.

Troitsky Club as a Whole

- **Preserved institutional knowledge:** Knowledge that would previously have left the club with graduating members is retained and remains accessible to future members, allowing the club's capabilities to accumulate over time instead of being rebuilt each year.

- **Sustainable software development:** The Toolbox is designed to be maintained by a software subteam within the club, creating a new way for members interested in software to contribute and ensuring the Toolbox continues to serve the club after the original development team has graduated.
- **Improved competitive performance:** Together, better access to past knowledge, analysis before construction, and consistent material data are expected to improve the performance of McMaster's teams at the Troitsky competition, benefiting the club's reputation as well as that of the McMaster Engineering Society and McMaster University.

G.4 Functionality Overview

High Level functional requirements for the toolbox are defined below:

- **Data Storage and Retrival.** Troitsky Toolbox must store past designs, materials testing informat, and relevant development information. Troitsky toolbox must be able to return relevant data quickly based on user searches. The system should allow for the addition of new information, streamilining the process of growing the repository.
- **Key Model Analysis Features and Bridge Predictions.** Provide a platform for teams to upload their bridge models through a standardized format, allowing the user to view and analyze the model, highlighting key metrics and predictions about the bridge's performance. This should decrease the barrier to entry to bridge analysis, allowing all members of a team to get involved in the analysis process, even without formal modelling knowledge. This will be integrated with the material testing support to provide more accurate bridge analysis from the stored material properties.
- **Materials Testing Support and Facilitated Result Analysis** As the McMaster Troitsky club begins to standardize knowledge collection, material testing data will be collected and stored in a standardized format. The service will streamline this process, allowing testing results to be easily uploaded and analyzed against past results, along with collecting supplementary testing information for future years.
- **Simplified and Comprehensive Knowledge Transfer** Support documentation from the McMaster Troitsky club through exit documents and other knowledge transfer methods, ensuring that information and designs are consolidated and accessible to future members. Specialized member roles, such as executive positions, will also be able to keep a record of their role responsibilities and key factors, to further improve the club organization.

G.5 High-level usage scenarios

- **Scenario 1: Repository Information Viewing** — User logs into web app and opens the knowledge repository, search for their desired information, open and view it.
- **Scenario 2: Knowledge Repository Addition** — User logs into web app and opens the knowledge repository and add the appropriate information to the repository.
- **Scenario 3: Transition Documentation Addition** — After logging into web app and being authenticated with the authority to edit knowledge transfer documents, the user opens and fill out their assigned transfer documents to be read by future executives.
- **Scenario 4: Performance Analysis** — User logs into web app and opens a STEP file either stored in the knowledge repository or that they just uploaded to the performance analysis system, then an analysis of their models load capacity and weak points is returned.
- **Scenario 5: Rule Compliance Verification** — User logs into web app and opens a STEP file either stored in the knowledge repository or that they just uploaded to the Rule Compliance system, then a overview of their compliance to the current years competition rules is returned.
- **Scenario 6: Materials Data Publication** — User logs into web app, after being authenticated as part of the materials testing team, they publish gathered materials data into the knowledge repository via the materials testing manager.

G.6 Limitations and exclusions

Below is a list of limitations for the system, items that are currently out of scope for development:

- The system is not looking to replace the more complex modelling capabilities of external/industry standard modelling software. Unlike the current apps being used by teams, such as Autodesk Inventor, SAP2000 and SolidWorks, the modelling component within Troitsky Toolbox will be intentionally simplified, which will leave many of the more advanced features out of scope.
- The system will not support user access for past members. The service will only be accessible by current members, with only the public-facing landing page and gallery available to others.
- The system will not replace all forms of communication between members. Given the accepted use of other platforms for communication and the ease of use in utilizing an existing app for communication, peer-to-peer or individual team channels will not be supported in the toolbox.

- The system will not replace registration into the club; initial applications into specific roles will remain external. Payment support will be excluded and done externally from the service with registration.

G.7 - Stakeholder and Requirements Sources

Table 2: Stakeholder Personas

Stakeholder	Persona	Category
McMaster Troitsky Executives	Mohammed Lee	Direct
General McMaster Troitsky Members	Tenun Grigoryan	Direct
Future Troitsky Members	Sui Smith	Direct
McMaster CAS Students	Kyle Levi	Direct
MES - McMaster Engineering Society	Mia Tanaka	Direct
Concordia University Troitsky Organization	Persona	Indirect
McMaster University	Persona	Indirect

G.7.1 - Direct Stakeholders

McMaster Troitsky Executives
This toolbox web service was requested by the Troitsky Executives; they have noticed issues with information carryover during executive transitions, as well as information and technical skill consolidation within the most senior teams. They would like a centralized application that can address these issues.

Persona: Mohammed Lee
Mohammed is a VP on the Troitsky Exec team. He has competed in Troitsky three times: twice as a team member and once as a coach. Through his experience on the team, he has developed a wealth of knowledge in both the administrative and technical sides of the club. He is graduating next year and is worried about the transition process for the new VP taking his place.

General Troitsky Members
General members will be the main users and the primary focus of our project. These users will both support and expand the data the system provides, and will be the main beneficiaries of the system.

Persona: Tenun Grigoryan
Tenun is a second-year member of Troitsky. He has competed on a team once before and is hoping to compete again this year. In his first year, there was a steep learning curve in the CAD design aspect of the competition, and he had to rely heavily on senior Troitsky members for tutorials and help. He hopes that this year, more resources will be provided to junior club members to decrease the barriers to competing.

Future Troitsky Members
Future Troitsky members are students coming to McMaster who will carry the torch of the Troitsky Club. This toolbox will lower the barrier to entry for new teams wanting to join Troitsky through both access to previous teams' work and documentation, as well as scripted load testing and weak-point analysis.

Persona: Sui Smith
Sui is a high school senior who has just been accepted into the McMaster Civil Engineering Program. She is part of student parliament at her school and competes in robotics competitions as one of her extracurriculars. She would love to continue competing on a technical team that aligns with her program. It goes without saying that she would prioritize joining a club that provides good resources for its introductory members.

McMaster CAS Students
Troitsky will need a software development team to maintain and extend the toolbox. This team will provide new extracurricular activities for CAS students at McMaster, allowing them to develop essential software engineering skills that will give them a leg up in school and work.

Persona: Kyle Levi
Kyle is a third-year CS student. He wants to work in large-scale system design and management when he graduates; he would like to join a technical team that focuses on maintaining and expanding a software system.

MES - McMaster Engineering Society
The MES organizes engineering clubs and seeks to benefit from the legitimization of the McMaster Troitsky Club through a centralized information platform. The MES also provides funding for the McMaster Troitsky Club, so the hosting of services and associated costs may be dependent on the given budget.

Persona: Mia Tanaka
Mia is the treasurer for the MES. She would like to ensure that the club funding she is approving is going to clubs that are investing in their long-term sustainability and development.

G.7.2 - Indirect Stakeholders

Concordia University Troitsky Organization
As Concordia University hosts the Troitsky competition, it establishes the rules and regulations of the competition and therefore influences what is prioritized by the direct stakeholders. This would influence the design of the system and which features would benefit teams the most.

McMaster University

The university itself is an indirect stakeholder, as it benefits from the success of the McMaster Troitsky Club and the success of teams during the Troitsky competition through an improved reputation as a school that produces capable students.

E Environment

E.2 Components

The system will interact with the following external components:

- **Troitsky Organizing Committee - Concordia:** As the Troitsky competition is organized at Concordia University by the Organizing Committee of the Engineering and Computer Science Association [**?**, Secs. 1, 13.1], much of what is valued in the product depends on what is valued in the competition itself. Between years, rules are often revised, and different competition components are added, removed or modified. The Organizing Committee also reserves the right to modify the rules at any time [**?**, Sec. 3.1]. As is the case, the product will be heavily dependent on the organization and must allow for compliance and rule checking in line with the current competition standards.
- **CAD Software Modelling Tools:** Design software, such as AutoCAD or SolidWorks, is used by teams to model their bridges and to create their STEP files [**?**], which will serve as the input for the Model Analysis.
- **Model Analysis Libraries:** For the modelling component of the product, external open-source libraries will be used to read and display the inputted STEP files and to perform their static analysis. This will be used to calculate/create the output predictions and model data (see P.5).
- **Hosting and Data Storage Platform:** The product will require a platform or server to host the website and the shared database. This is where the repository of information, uploaded files and backups will be stored.
- **Materials Testing Department Setup:** As the product seeks to support the material testing department, the app will need to work around the data formats and the processing the team prefers.
- **Club Sponsors and University Funding:** The MES and external sponsors dictate the funding that the club receives yearly. This funding may dictate the future upkeep of the product, being required for the hosting and data storage platform.

E.3 Constraints

Troitsky Toolbox Constraints

- **Competition Rules:** The model compliance checker and other support services within Troitsky Toolbox must comply with the rules set out for the competition [**?**]. Outdated guides must be labelled to ensure users understand they do not fit with current competition rules.

- **Member Data and Privacy:** As the service seeks to compile team and member information, member privacy and sharing permissions dictate what can be collected. During the current competition year, individual team files must be left separate, with only executives and coaches being able to view all files. After the competition is complete, exit documentation collected will determine what individuals wish to have stored for future years.

- **Club Data Format Compatibility :** The system must accommodate the file formats that the club utilizes, being constrained to a generalized file format when multiple are available. In the case of modelling files, STEP (.stp) files must be used as input as they are available across utilized services. The material testing data collection must accept .csv files, working with the data exporting the club utilizes.

Operational Constraints

- **Budget:** The funding set out by the MES, along with the yearly club sponsorships, will determine the budget and therefore constrain the services and upkeep of the product. Troitsky Toolbox must operate within its share of the overall club funds, dictating the hosting and storage capabilities of the service.

- **Troitsky Software Subteam:** Maintenance year to year after this year will be limited by the number of software subteam members the club recruits. As the subteam is made up of student volunteers, the experience and drive of the members will also determine the level of development of the system.

E.4 Assumptions

The following properties of the environment are assumed.

- **Competition Format Stability:** Despite the product being adaptable to changes within rules and smaller operational changes, the overall format of the Troitsky competition is assumed to remain stable across years. While some forms of bridges, materials or scoring are revised, the product serves the standard competition format of bridges made only of popsicle sticks, toothpicks, white glue and dental floss, loaded at mid-span until failure [**?**, Secs. 7.1, 10.5]. The rule set may change across years, as shown

by the summary of changes from the previous edition in each rulebook. Competition rulebooks are assumed to be published before each competition, allowing for integration into the rule compliance checker once yearly, with further updates as revisions are released.

- **User Access and Device Availability:** As the product will be available through a website, there is an assumption that members will both have access to a suitable device (desktop or laptop) and a standard internet browser. There is also an assumption that all users will have a working internet connection when accessing the service.
- **McMaster Troitsky Club Organizational Structure:** The role-based access control of the system will be designed around the current club structure, specifically around the roles in the organizational body of the club (general members, team captains, coaches, executives). It is assumed that this structure will not change frequently and that the access of each role will remain constant across competition years.
- **Continued Club Operation:** There is an assumption that the Troitsky competition, both as a whole and as McMaster's individual club, will continue to operate. This includes the assumption that teams will continue to register and that the software subteam will continue to have registered members.
- **Model File Conventions:** Bridge models are assumed to be exported in the STEP file format. This export will be in line with a standard export format outlined by the club, allowing for some consistency in how bridge members and units are represented across files. In line with P.6, these file conventions are linked to the most complex and high-risk component of the product, so an agreed-upon standard simplifies the scope of the service, serving as a mitigation strategy for the risk.
- **Material Behaviour:** There is an assumption of some consistency in material properties while the club purchases materials from the same supplier. As the model analysis predictions may rely on the measured material properties of popsicle stick and glue slabs, constant material properties are a required assumption to create any form of performance prediction.

E.5 Effects

- Centralization of development information - the wealth of development information (e.g. past designs, material testing information) will be in one central location for future members to access, which will facilitate onboarding.
- Model analysis - team members will be able to submit their models and recieve key metrics and predictions regarding that model through an automatic analysis.

- Facilitated comparison - team members will be able to easily compare modelling results for different models. This will facilitate development because team members will be able to start from the best and abandon changes that caused the model to degrade.
- Simplified knowledge transfer - veteran team members will be able to share their vast knowledge with new members more easily. This is critical, especially for outgoing team members.
- Creation of a permanant software sub-team - this project will be the genesis of a permanant sofware subteam of the club. Furthermore, brief but top-tier resources alongside the software suitewill be developed to smooth the onboarding of new software subteam members.

E.6 Invariants

- The knowledge entered into the system, including development knowledge, scientific and engineering methods, and veteran wisdom, shall be fully preserved.
- Any model entered into the system, whether developed before or after the system's released, shall be fully preserved and not corrected at all to preserve development history as faithfully as possible for easy comparison.
- The rules of the competition that are relevant to the system shall be fully preserved and not be mutated at all. This is because modelling will be based of the rules as they are, and any deviation from the rules will result in inferior models.

S System

S.1 Components

S.1.1 Component Descriptions

The Troitsky Toolbox is a web-based application structured around a central broker. Each tool, as well as the shared data storage, is a service that registers with the broker. All communication between the user interface and the services, and between the services themselves, passes through the broker, so that no service depends directly on another. The system relies on the following software components:

- **Web Server:** The component in charge of hosting the Toolbox Web App and serving it to members' browsers. It receives all incoming requests (`Handle Request`), authenticates members at login through the Access Control System, and forwards requests for club services to the Toolbox Broker. It also serves the public club landing page, which requires no authentication.

- **Access Control System:** The component in charge of managing member accounts and their roles (`Manage Members`), such as general member, executive, and software subteam maintainer. It also records which team, if any, each member belongs to. It authenticates users (`Authenticate`) and decides which services and data each member may access based on their role and team (`Authorize`), ensuring that club services and stored data are limited to authorized members, and that a team's in-progress work is limited to that team.

- **Toolbox Broker:** The component in charge of coordinating all communication within the system. Services register with the broker (`Register Service`), advertising the operations they provide. The broker receives each request (`Route Request`), verifies it with the Access Control System (`Authorize`), and forwards it to the service that provides the requested operation. New tools can be added to the Toolbox by registering them with the broker, without modifying the broker or any existing service.

- **Knowledge Repository:** The service in charge of the centralized, persistent store of all club information (`Manage Data`), including documents, current and past bridge models, material test results, competition rules, and handoff documents. Other services store and retrieve their data by sending requests through the broker, which allows information produced by one tool to be found and reused by members and by other tools. It also allows members to search, filter, and contribute content (`Search Content`).

- **Model Analysis System:** The service in charge of importing bridge models in a standardized 3D format (`Upload Model`), rendering an interactive visualization of the model, and extracting the geometric and structural data of the bridge (`Model Data`). It works with two kinds of bridge models, both stored in the Knowledge Repository:

  - **Current team models:** The models a team is developing for the upcoming competition. These are accessible only to the members of that team, so that teams can work on their designs independently.
  - **Past designs:** Models of bridges from previous years, which any member can load into the model viewer to inspect and learn from, but cannot modify. At the end of each competition season, a team's current models become past designs.

  Its analysis capabilities are provided by two subsystems, which extend its functionality while remaining separate from it and from each other:

  - **Performance Analysis System:** Computes key metrics and predictions about a bridge's expected performance (`Analyze Performance`), using the extracted model data and the material properties provided by the Material Testing Manager, both requested through the broker.

    - **Rule Compliance System:** Assesses a bridge model against the current Troitsky competition rules (`Check Compliance`), using the extracted model data requested through the broker. It maintains the set of rules being checked (`Manage Rules`) so that they can be updated as the competition rules change from year to year.

- **Material Testing Manager:** The service in charge of facilitating the storage and access of material testing results in a standardized format (`Manage Test Results`), and analyzing new results against those collected in previous years. This service also derives material-specific data from accepted test results (`Material Properties`), which is provided to the Performance Analysis System through the broker.

- **Knowledge Transfer Manager:** The service in charge of collecting and organizing exit documents, role records, and onboarding material (`Manage Handoff Documents`), so that the knowledge held by graduating members and executive positions is preserved for future members.

In addition to these components, the system will interact with external components, such as a CAD/geometry library used by the Model Analysis System to read STEP files (see E.2 for details).

Club members interact with the system through the **Toolbox Web App**, a browser-based interface served by the Web Server, which presents each of the tools above. Through it, members can browse and contribute to the repository, upload and analyze bridge models, record material tests, and complete handoff documents. Members of the software subteam use the same interface, with elevated privileges granted by the Access Control System, to maintain accounts, competition rules, and stored content.

Figure 1 shows the components of the system and the interfaces through which they interact.

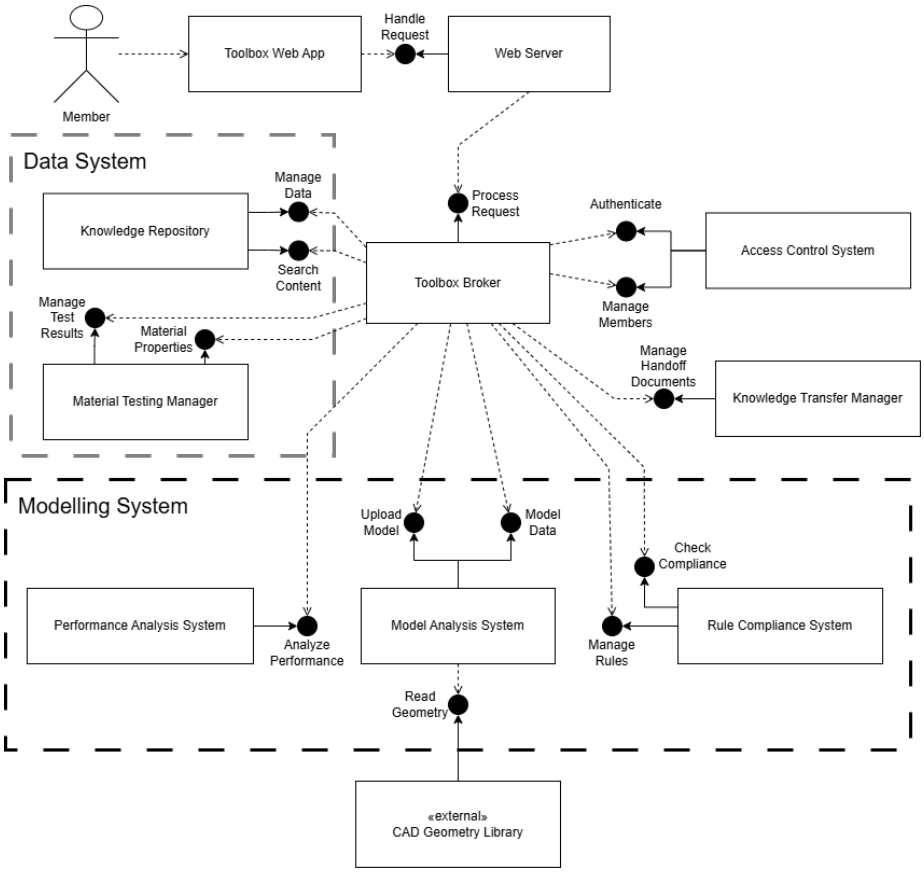


Figure 1: Component diagram of the Troitsky Toolbox. All communication between services passes through the Toolbox Broker.

S.1.2 Component Inputs and Outputs

Component	Inputs	Outputs
Web Server	• Incoming web requests from members' browsers • Authentication decisions from the Access Control System • Responses and data routed from the Toolbox Broker	• Served Toolbox Web App and public landing page • Authentication requests to the Access Control System • Forwarded club service requests to the Toolbox Broker
Access Control System	• Login credentials/Authentication requests • Authorization requests (verifying service/data access) • Member account, team, and role management data	• Authentication decisions (success/failure) • Authorization decisions based on roles and teams • Member and team registry information
Toolbox Broker	• Service registration requests • Incoming service requests from the Web Server and other services • Authorization decisions from the Access Control System • Responses/data from target services	• Authorization requests to the Access Control System • Routed requests to the appropriate registered service • Routed responses back to the Web Server or requesting service
Knowledge Repository	• Content to be stored (documents, bridge models, test results, rules, etc.) • Search queries and filter parameters • Data retrieval requests from other services via the broker	• Stored club information and files • Filtered search results • Acknowledgment of successfully stored content

Model Analysis System	• 3D bridge models (e.g., STEP files) • Material properties (from Material Testing Manager) • Competition rules (via Knowledge Repository/Broker)	• Interactive 3D model visualizations • Extracted geometric and structural data • Performance metrics and predictions • Rule compliance assessments
Material Testing Manager	• Raw material test results submitted by members • Requests for material properties from other services (e.g., Model Analysis System)	• Standardized material test records • Derived material-specific properties and data
Knowledge Transfer Manager	• Submitted exit documents and role records • Submitted onboarding materials • Retrieval requests for handoff documents	• Organized handoff documents and knowledge records • Preserved historical executive and subteam data

S.2 Functionality

This section describes the behaviour of each component identified in S.1. Each subsection lists the functional and non-functional requirements of one component. Requirements are labelled with a prefix identifying their component, followed by a number: for example, FR-BR-1 is the first functional requirement of the Toolbox Broker, and NFR-BR-1 its first non-functional requirement. The prefixes are WS (Web Server), AC (Access Control System), BR (Toolbox Broker), KR (Knowledge Repository), MA (Model Analysis System), PA (Performance Analysis System), RC (Rule Compliance System), MT (Material Testing Manager), KT (Knowledge Transfer Manager), and SYS (system-wide requirements). The symbolic constants used by the non-functional requirements are defined in Table 19 (Appendix A.1).

Requirements whose reason is not obvious are followed by a rationale. Rationales refer to the two sources of requirements described in P.7: the member survey (P.7.1, $n = 20$), whose feature ratings are summarized in Table 18, and discussions with the Troitsky executives involved in the project (P.7.2). The rationale for each numeric value is given with its constant in Appendix A.1.

S.2.1 Web Server

Functional Requirements

1. **Public landing page:** Web Server shall serve a public club landing page to any visitor, without requiring authentication. (FR-WS-1)
   Rationale: Club announcements and general information (survey feature K) were rated useful or better by most respondents (mean 3.85, 14 of 20 rating it "Very useful" or higher). Leaving the page public also lets prospective members learn about the club before they have an account.

2. **Login and logout:** Web Server shall allow members to log in by submitting their credentials to the Access Control System, and to log out, ending their session. (FR-WS-2)

3. **Request forwarding:** Web Server shall forward every request for a club service from an authenticated member to the Toolbox Broker, along with the identity of the member making the request. (FR-WS-3)
   Rationale: The broker architecture (S.1) gives every service a single entry point, so no service is reachable from the browser without passing access control. The member's identity is forwarded because authorization depends on the member's role and team (FR-AC-3).

Non-Functional Requirements

1. **Serve web app:** Web Server shall serve the Toolbox Web App to members' browsers. (NFR-WS-1)

2. **Encrypted communication:** Web Server shall only communicate with browsers over an encrypted connection (HTTPS). (NFR-WS-2)

Rationale: Members' passwords and unreleased team designs are transmitted between the browser and the server. Without encryption, they could be read by anyone on a shared network, such as campus Wi-Fi.

3. **Session expiry:** Web Server shall end a member's session after SESSION_TIMEOUT (30 minutes) of inactivity, so that an unattended browser does not leave club data accessible. (NFR-WS-3)

S.2.2 Access Control System

Functional Requirements

1. **Authentication:** Access Control System shall verify the credentials submitted by a member, and shall reject login attempts with invalid credentials or for deactivated accounts. (FR-AC-1)

2. **Roles:** Access Control System shall assign each account exactly one role: general member, team member, team captain, club executive, or software subteam maintainer. (FR-AC-2)
Rationale: The roles follow the club's structure, as described by the Troitsky executives (P.7.2) and confirmed by the roles reported in the member survey (P.7.1). Each role needs a different set of permissions: captains manage their own team's data, executives manage the club and its rules, and maintainers administer the system. Assigning exactly one role keeps the permission check unambiguous.

3. **Authorization:** Access Control System shall determine, for each request it is asked to verify, whether the current role and team of the requesting member permit the requested operation. (FR-AC-3)

4. **Member management:** Access Control System shall allow executives and maintainers to create accounts, deactivate accounts, and change the role of an account, so that membership can be updated as members join, are promoted or demoted, graduate, or otherwise leave. (FR-AC-4)

5. **Team membership:** Access Control System shall allow executives and maintainers to assign each account to at most one team for the current competition year, and to change or remove that assignment. (FR-AC-5)
Rationale: Teams within the club compete against each other, so a member belonging to two teams could see a rival team's design. Allowing the assignment to change supports members switching teams, or teams reforming, during the year.

6. **Team data isolation:** Access Control System shall deny any member who is not on that team, including former members, access to that team's current bridge models and analysis results. (FR-AC-6)
Rationale: A private workspace for each team (survey feature I) was rated "Very useful" or higher by 11 of 20 respondents. Because teams compete, a current design must stay private until the season ends, when it

becomes a past design visible to all members (FR-MA-6). Former members are included so that a member who changes teams cannot take access to their old team's design with them.

Non-Functional Requirements

1. **Credential security:** Access Control System shall store passwords only as salted hashes produced by an industry-standard algorithm [**?**], and shall never store passwords in plain text. (NFR-AC-1)
   Rationale: Students commonly reuse passwords across services, including their university accounts. Hashing ensures that a leak of the Toolbox database does not expose members' passwords, as recommended by the OWASP Password Storage Cheat Sheet [**?**].

2. **Effective role alterations:** Access Control System shall ensure that if a user's role is altered, the change shall be effective by the user's next request to the system. (NFR-AC-2)
   Rationale: When a member is demoted, deactivated, or moved to another team, a delay before the change takes effect would let them keep accessing data they are no longer permitted to see, which would break team data isolation (FR-AC-6).

S.2.3 Toolbox Broker

Functional Requirements

1. **Service registration:** Toolbox Broker shall allow services to register at system startup, recording the operations that each registered service provides. (FR-BR-1)
   Rationale: Registration lets the broker discover services instead of having them hard-coded. This is what makes the extensibility requirement NFR-BR-1 achievable.

2. **Request authorization:** Toolbox Broker shall verify every request with the Access Control System before forwarding it, and shall reject requests that are not authorized. (FR-BR-2)
   Rationale: Checking authorization once, in the broker, means each service does not have to implement its own access control, and a new service is protected as soon as it is registered.

3. **Request routing:** Toolbox Broker shall forward each authorized request to the registered service that provides the requested operation, and shall return that service's response to the requester. (FR-BR-3)

4. **Unavailable services:** Toolbox Broker shall return an error message to the requester when the requested operation is not provided by any registered service, or when the service providing it does not respond. (FR-BR-4)

Rationale: An explicit error lets the member know a feature is temporarily unavailable, instead of the page hanging. It also lets the rest of the system keep working (NFR-SYS-3).

Non-Functional Requirements

1. **Extensibility:** Toolbox Broker shall allow a new service to be added to the system by registering it, without modifying the source code of the broker or of any existing service [**?**]. (NFR-BR-1)
   Rationale: The Toolbox will be maintained and expanded by a club software subteam whose members change every year (P.7.2). Adding a feature, such as onboarding material (survey feature H), which is not part of the initial scope, should not require future developers to understand or risk breaking the existing services.

2. **Routing overhead:** Toolbox Broker shall add no more than MAX_BROKER_OVERHEAD (100 ms) to the response time of any request it routes. (NFR-BR-2)
   Rationale: Every request passes through the broker, so its overhead is added to every response time in this section.

3. **Request logging:** Toolbox Broker shall log each request it handles, including the requesting member, the requested operation, the time, and whether the request succeeded. (NFR-BR-3)
   Rationale: Future maintainers will not have the original developers to ask, so logs are their main tool for diagnosing problems. Logs also let executives check who accessed a team's data if a privacy concern is raised.

S.2.4 Knowledge Repository

Functional Requirements

1. **Data storage:** Knowledge Repository shall store and return the data of every other service, including documents, bridge models, material test results, competition rules, and handoff documents. (FR-KR-1)
   Rationale: The survey found the club's data spread across Microsoft Teams (13 of 20), Google Drive, Discord, email, and individual members' devices (P.7.1). Lack of access to past designs and reference work was the most common frustration reported. One shared store is what allows knowledge to outlast the members who created it.

2. **Content upload:** Knowledge Repository shall allow members to upload files of any common type, along with descriptive metadata including title, category, year, team, and tags. (FR-KR-2)
   Rationale: Club material comes in many formats, including CAD models, spreadsheets, slides, and photos of joints, so the repository does not limit file types. The metadata fields are the ones members need to filter by (FR-KR-3). Without them, uploaded files would be as hard to find as they are in the current shared drives.

3. **Search and filter:** Knowledge Repository shall allow members to search stored content by keyword across titles and metadata, and to filter results by category, year, team, and tag. (FR-KR-3)
   Rationale: A searchable archive of past designs (survey feature A) had a mean rating of 4.35 and was chosen in the top 3 by 11 of 20 respondents, the second-highest priority. Joint construction data was the most valued type of past information (P.7.1).

4. **View and download:** Knowledge Repository shall allow members to preview supported file types in the browser, and to download any stored file. (FR-KR-4)

5. **Content management:** Knowledge Repository shall allow the uploader of an item, and executives, to edit its metadata or archive it. Archived items shall remain retrievable, but shall be excluded from default search results. (FR-KR-5)
   Rationale: 17 of 20 respondents want their team's work preserved for future teams (P.7.1). Archiving instead of deleting keeps outdated material out of the way without permanently losing it. Editing is limited to the uploader and executives so that one member cannot alter another's contribution.

Non-Functional Requirements

1. **Search responsiveness:** Knowledge Repository shall return search results within MAX_SEARCH_TIME (2 seconds) for a repository containing up to MAX_REPOSITORY_ITEMS (10,000 items). (NFR-KR-1)

2. **Data durability:** Knowledge Repository shall be backed up at least every BACKUP_INTERVAL (7 days), so that no more than BACKUP_INTERVAL of stored data can be lost in the event of a failure. (NFR-KR-2)
   Rationale: Once the repository replaces the club's scattered storage, it becomes the only copy of much of the club's history, so a single failure must not be able to erase it.

S.2.5 Model Analysis System

Functional Requirements

1. **File validation:** Model Analysis System shall reject files that are not valid STEP files, or that contain no readable geometry, and shall display an error message describing the problem. (FR-MA-1)
   Rationale: Analysing an invalid or empty model would produce meaningless estimates and compliance results that members might act on. Rejecting the file with a clear reason lets the member fix and re-export it.

2. **3D visualization:** Model Analysis System shall render an interactive 3D view of an uploaded model, which the member can rotate, zoom, and pan. (FR-MA-2)

Rationale: The model viewer (survey feature F) was the lowest-priority feature on its own (mean 3.55, no top-3 picks). It is kept because members need it to confirm that the system read their model correctly before trusting its analysis, and to inspect past designs (FR-MA-5), which is a top priority.

3. **Data extraction:** Model Analysis System shall extract from each uploaded model the geometric and physical properties of the bridge required by its analysis subsystems, such as its dimensions and mass. (FR-MA-3)
   Rationale: A bridge weight estimator (survey feature C) had a mean rating of 4.10 and was chosen in the top 3 by 8 respondents. Extracting the data once, in one place, gives the performance and rule compliance subsystems consistent inputs.

4. **Current model storage:** Model Analysis System shall store each uploaded model and its extracted data in the Knowledge Repository as a current model of the uploading member's team, accessible only to members of that team, so that it can be reopened and analyzed later. (FR-MA-4)
   Rationale: Members reported difficulty collaborating on a shared CAD model (P.7.1). Storing current models with the team lets every team member reach the latest version, while team data isolation (FR-AC-6) keeps it from rival teams.

5. **Past design viewing:** Model Analysis System shall allow any member to browse past designs, filtered by year and team, and to load any past design into the model viewer and its analysis subsystems for inspection, without being able to modify it. (FR-MA-5)
   Rationale: Access to past designs was the most common frustration in the survey, and the archive of past designs (feature A) is the second-highest priority. Past designs are read-only so that the historical record, and its link to recorded competition results, cannot be changed.

6. **Archiving current models:** Model Analysis System shall allow executives to convert all current team models into past designs at the end of a competition season, recording the competition year and team of each. (FR-MA-6)
   Rationale: Designs only need to be private while teams are competing. Converting them at the end of the season, as a single executive action, ensures that every team's work enters the archive, which 17 of 20 respondents want, without relying on each team to remember to share it.

Non-Functional Requirements

1. **Model upload:** Model Analysis System shall allow members to upload bridge models in the STEP (`.stp`/`.step`) format, which is the only model format accepted by the system. (NFR-MA-1)
   Rationale: STEP is an open ISO standard (ISO 10303) that the CAD tools members use can export, so a single format supports every team

regardless of their software. All 20 survey respondents have CAD experience (P.7.1). Accepting one format also means only one parser has to be maintained (P.5, Open CASCADE).

2. **Load time:** Model Analysis System shall display the 3D visualization of a typical club bridge model within MAX_MODEL_LOAD_TIME (30 seconds) of upload or of being selected. (NFR-MA-2)

3. **Extraction accuracy:** Model Analysis System shall extract dimensions that match those of the source model to within DIMENSION_TOLERANCE (0.5 mm). (NFR-MA-3)
   Rationale: Extracted dimensions feed rule compliance checks and mass estimates. An inaccurate extraction could pass a bridge that violates a size rule.

S.2.5.1 Performance Analysis System

Functional Requirements

1. **Failure load estimate:** Performance Analysis System shall estimate the failure load of a bridge model, being the maximum load the bridge can support before failure [**?**, Sec. 10.5]. (FR-PA-1)
   Rationale: Failure load prediction (survey feature E) is the highest-priority feature in the survey: mean 4.35, 18 of 20 rating it "Very useful" or higher, and 12 top-3 picks. Members reported that they "don't know what works until we build it and it fails at competition" (P.7.1).

2. **Load-to-mass ratio:** Performance Analysis System shall compute the estimated load-to-mass ratio of a bridge model, from its estimated failure load and its extracted mass [**?**, Sec. 10.6]. (FR-PA-2)
   Rationale: Structural efficiency (survey feature D) is the third-highest priority (mean 4.25, 10 top-3 picks). Load-to-mass ratio lets teams compare designs of different sizes, not just the strongest.

3. **Material selection:** Performance Analysis System shall use the material properties provided by the Material Testing Manager, and shall allow the member to choose which set of material data, such as a given year or material batch, to apply. (FR-PA-3)
   Rationale: The strength of popsicle sticks, glue, and floss varies between suppliers and years. Using the club's own measured data (S.2.6), for the batch actually being built with, gives more realistic estimates than generic textbook values.

4. **Assumption disclosure:** Performance Analysis System shall display the assumptions and limitations of each estimate alongside its results, so that members do not mistake an estimate for a guaranteed result. (FR-PA-4)
   Rationale: 13 of 20 respondents have two years or less in the club, and 9 have CAD experience only from courses (P.7.1). Less experienced members are the most likely to treat a number from the system as exact.

Showing the limitations (NFR-PA-1) reduces the risk of a team building an unsafe design because of an over-trusted estimate.

Non-Functional Requirements

1. **Estimate accuracy:** For past club bridges whose tested failure loads are recorded, Performance Analysis System shall estimate the failure load to within FAILURE_LOAD_TOLERANCE (25%) of the recorded value in competition. (NFR-PA-1)
   Rationale: Recorded competition results of past club bridges are the only real failure data available, so accuracy is measured against them.

2. **Computation time:** Performance Analysis System shall produce results for a typical club bridge model within MAX_ANALYSIS_TIME (30 seconds). (NFR-PA-2)

3. **Extensibility:** Performance Analysis System shall be structured so that additional performance metrics can be added in the future without modifying the computation of existing metrics [**?**]. (NFR-PA-3)
   Rationale: The initial metrics cover the highest-priority survey features (D and E). Members also reported running "all the different structural analysis tests" as a frustration (P.7.1), so future software subteams are expected to add further analyses, such as weak point identification, without the risk of changing validated results.

S.2.5.2 Rule Compliance System

Functional Requirements

1. **Rule import:** Rule Compliance System shall allow executives and maintainers to import a set of competition rules for a given competition year. (FR-RC-1)
   Rationale: The Troitsky competition rules are revised every year, so the rule set cannot be fixed in the system. Storing rules by year also allows past designs to be checked against the rules they were built for.

2. **Rule configuration:** Rule Compliance System shall allow executives and maintainers to create, edit, and remove individual rules, each defined by the property it constrains, such as a dimension or mass, and its allowed value or range. (FR-RC-2)
   Rationale: Most measurable competition rules are limits on a single property. Representing them in this form lets them be checked automatically, and changed one at a time when the rules are revised.

3. **Compliance check:** Rule Compliance System shall evaluate a bridge model against the active rule set, using the data extracted by the Model Analysis System, and shall report a pass or fail result for each rule. (FR-RC-3)

Rationale: A competition rules reference (survey feature J) had a mean rating of 4.00 and 5 top-3 picks. Checking the model against the rules before construction catches violations while they are still cheap to fix (G.3).

4. **Violation details:** For each failed rule, Rule Compliance System shall report the violated rule, the measured value, and the allowed value or range. (FR-RC-4)
   Rationale: A bare "fail" does not tell the team how much to change the design. Showing the measured and allowed values tells them the size of the correction needed.

5. **Manual checks:** Rule Compliance System shall list the rules that cannot be verified from the model, such as restrictions on materials or construction methods, as manual checks for the team to confirm. (FR-RC-5)
   Rationale: A STEP model contains geometry, not information about how the bridge was built. Listing these rules explicitly prevents a team from reading an all-pass result as full compliance.

Non-Functional Requirements

1. **Check correctness:** Rule Compliance System shall report no false passes on a test suite of models with known rule violations. (NFR-RC-1)
   Rationale: A false pass and a false fail have very different costs. A false fail costs the team a manual double-check, while a false pass can lead to a bridge being built and then disqualified at competition. False passes are therefore not tolerated.

2. **Rule updatability:** Rule Compliance System shall allow an executive without programming knowledge to configure the rule set for a new competition year without modifying source code. (NFR-RC-2)
   Rationale: Rules change every year, but the software subteam may not be available when they are released. Executives come from many engineering disciplines and, as discussed with the Troitsky executives (P.7.2), cannot be assumed to know how to program.

S.2.6 Material Testing Manager

Functional Requirements

1. **Result entry:** Material Testing Manager shall allow members to record individual test results through a form, including the test type, material, specimen dimensions, applied load, date, and tester. (FR-MT-1)
   Rationale: A database of material test results (survey feature B) has the highest mean rating of any feature (4.45, with 18 of 20 rating it "Very useful" or higher). Recording the specimen dimensions, date, and tester with each result makes results comparable across years and lets questionable results be traced back to how they were produced.

2. **Input validation:** Material Testing Manager shall reject test results that are missing required fields, or that contain physically impossible values such as negative dimensions or loads, and shall identify the invalid rows or fields. (FR-MT-2)
   Rationale: Accepted results feed the material properties used in failure load estimates (FR-MT-4, FR-PA-3), so a single typing error could distort every estimate that uses them.

3. **Historical comparison:** Material Testing Manager shall compare new test results against results from previous years, displaying summary statistics (mean and standard deviation) and plots. (FR-MT-3)
   Rationale: Comparing a new batch against previous years shows whether this year's materials are weaker or stronger than usual, and highlights results far from the norm that may come from a testing error.

4. **Material properties:** Material Testing Manager shall compute aggregated material properties from accepted test results, and shall provide them to the Performance Analysis System. (FR-MT-4)

5. **Result review:** Material Testing Manager shall allow executives to flag test results as invalid, excluding them from aggregated material properties without deleting them. (FR-MT-5)
   Rationale: Some errors, such as a badly prepared specimen, pass input validation. Flagging instead of deleting keeps a record of what was excluded and why, which traceability (NFR-MT-2) requires.

Non-Functional Requirements

1. **Result import:** Material Testing Manager shall allow members to import material test results from CSV and Excel (`.xlsx`) files that follow a standardized template. (NFR-MT-1)
   Rationale: Testing is done in batches, and results are typically recorded in spreadsheets during testing. Most teams already work in the Microsoft 365 suite (13 of 20 use Microsoft Teams, P.7.1), so importing their existing spreadsheets avoids typing results in again. A standard template allows the import to be validated.

2. **Traceability:** Material Testing Manager shall ensure that every aggregated material property can be traced to the individual test results it was computed from. (NFR-MT-2)
   Rationale: When an estimate disagrees with a real test, members need to be able to check whether the material data behind it was sound.

3. **Export:** Material Testing Manager shall allow stored test results to be exported in CSV format, so that the data remains usable outside the system. (NFR-MT-3)
   Rationale: Members may want to analyse the data with their own tools, and the club's data should not be locked into the Toolbox if the system is ever replaced.

S.2.7 Knowledge Transfer Manager

Functional Requirements

1. **Exit document templates:** Knowledge Transfer Manager shall provide an exit document template for each executive position and for team leads. (FR-KT-1)
   Rationale: Exit and handover documents (survey feature G) were rated "Very useful" or higher by 15 of 20 respondents. Ease of transition between executives is a main goal identified with the Troitsky executives (P.7.2). A template for each position prompts the outgoing member for the information their successor will need, instead of leaving them to decide what to write.

2. **Exit document submission:** Knowledge Transfer Manager shall allow members to complete, save as drafts, and submit exit documents. (FR-KT-2)
   Rationale: Exit documents are written near the end of the year, at the same time as exams, so members need to be able to write them over several sessions.

3. **Role records:** Knowledge Transfer Manager shall maintain a record for each executive position, describing its responsibilities, key contacts, and yearly timeline, which the current holder of the position can update. (FR-KT-3)
   Rationale: An exit document records one year. A role record is kept up to date across years, so the incoming executive receives a current description of the role, not only their predecessor's account.

4. **Publication:** Knowledge Transfer Manager shall store submitted exit documents and role records in the Knowledge Repository, where they can be searched like other content. (FR-KT-4)

5. **Outstanding documents:** Knowledge Transfer Manager shall allow executives to view which executives and team leads have not yet submitted their exit documents. (FR-KT-5)
   Rationale: Exit documents are only useful if they are written before the member leaves the club. Tracking which are outstanding lets executives follow up while the outgoing member is still available.

Non-Functional Requirements

1. **Findability:** Knowledge Transfer Manager shall allow a new member to locate the role record for a given executive position within MAX_ROLE_SEARCH_TIME (2 minutes), as measured in usability testing. (NFR-KT-1)
   Rationale: Role records only reduce the support burden on previous executives (G.3) if a new executive can find them without help.

2. **Longevity:** Knowledge Transfer Manager shall allow exit documents and role records to be exported to PDF, so that they remain accessible independently of the system. (NFR-KT-2)
   Rationale: Handover records are meant to last many years, possibly longer than the Toolbox itself or its hosting. PDF is a widely supported format that executives can also share with people who do not have accounts.

S.2.8 System-Wide Requirements

Non-Functional Requirements

1. **Browser support:** The Toolbox Web App shall be fully functional on the current versions of Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari on Windows, Macintosh, or Linux personal computers. (NFR-SYS-1)
   Rationale: These four browsers cover the computers members use. Mobile devices are excluded because the main tasks, such as uploading CAD models, 3D viewing, and entering test data, are done at a computer where the CAD files are stored.

2. **Capacity:** The system shall support at least MIN_CONCURRENT_USERS (50 members) using it concurrently without exceeding the response times specified in this section. (NFR-SYS-2)

3. **Service independence:** The failure of any single service shall not prevent members from using the remaining services. (NFR-SYS-3)
   Rationale: The services are independent in purpose: a member searching the archive does not need the performance analysis to be running. The analysis services are also the most complex and the most likely to fail, so isolating them protects the services every member relies on.

4. **Maintainability:** The system shall be documented well enough that a new software subteam member, with experience in C# and web development, can set up a local development environment and deploy a change within MAX_ONBOARDING_TIME (1 week). (NFR-SYS-4)
   Rationale: Maintainability and expandability by future software subteams is the main long-term challenge of the project (P.7.2). Once the original developers graduate, the documentation is the only handover the next subteam receives.

S.3 Interfaces

This system requires several software interfaces, including:

- An RESTful API (application programming interface), which shall enable users to communicate with the backend through the frontend and alter it when permitted and desired.

- External static modelling libraries, which shall perform technical engineering calculations on given inputs, both on calculations necessary to score STP models and inputs for materials testing.
- A robust external server which holds the entire swath of data this toolbox relies on, including STP files, material testing inputs, documents containing non-technical club knowledge and roles for user accounts.
- Some aid to login. This may be an entire external login service or merely a cryptographic hash function which guards passwords to minimize collateral damage in case of a data leak.

S.4 Detailed usage scenarios

S.4.1 – Knowledge Repository Information Viewing

Use Case Name: Knowledge Repository Information Viewing
Primary Actor: User
Precondition:

- User has an account on the Troitsky Toolbox web application with general member permissions.

Trigger: A Troitsky member would like to view data stored in the Knowledge Repository.
Main Success Scenario:

1. User logs into the web application and is successfully authorized by the Access Control System.
2. User requests to open the Knowledge Repository via the Toolbox Broker.
3. User requests the information they are interested in from the Knowledge Repository via the Toolbox Broker.
4. Toolbox Broker returns the requested information.
5. User views, and optionally downloads, the information.

Secondary Scenarios:

- Access Control System fails verification.
    1. User is unable to log into the web application.
- Toolbox Broker fails at some point.
    2. Request to open the Knowledge Repository fails.
    3. Search request in the Knowledge Repository fails.
    4. Toolbox Broker fails to return the requested information.

- Information is corrupted.

    5. User cannot view the information.

Success Postcondition: User was able to find and access the information they were searching for.

Justification: This scenario describes a user's ability to read from the Knowledge Repository, directly aligning with FR-KR-1 and FR-KR-3.

S.4.2 – Knowledge Repository Information Addition

Use Case Name: Knowledge Repository Information Addition
Primary Actor: User
Precondition:

- User has an account on the Troitsky Toolbox web application with general member permissions.

Trigger: A Troitsky member would like to add relevant data to the Knowledge Repository.
Main Success Scenario:

1. User logs into the web application and is successfully authorized by the Access Control System.
2. User requests to open the Knowledge Repository via the Toolbox Broker.
3. User requests to upload information to the Knowledge Repository via the Toolbox Broker.
4. User specifies the data type of the information and uploads it to the Knowledge Repository.

Secondary Scenarios:

- Access Control System fails verification.

    1. User is unable to log into the web application.

- Toolbox Broker fails at some point.

    2. Request to open the Knowledge Repository fails.
    3. Upload request to the Knowledge Repository fails.

Success Postcondition: The information the user wants to store is successfully uploaded to the Knowledge Repository.

Justification: This scenario describes a user's ability to write to the Knowledge Repository, directly aligning with FR-KR-2.

S.4.3 – Transition Information Addition

Use Case Name: Transition Information Addition
Primary Actor: Troitsky Executives
Precondition:

- User has executive access to the Troitsky Toolbox web application.

Trigger: A Troitsky executive wants to add to their transition documents.
Main Success Scenario:

1. User logs into the web application and is successfully authorized by the Access Control System.
2. User requests to open the Knowledge Transfer Manager.
3. Toolbox Broker confirms with the Access Control System that the user has the authority to access the Knowledge Transfer Manager.
4. Toolbox Broker directs the user to the Knowledge Transfer Manager.
5. User selects the documents they would like to fill out or edit.
6. User fills out or edits the documents and saves them.
7. Toolbox Broker saves the transition documents to the Knowledge Repository.

Secondary Scenarios:

- Access Control System fails verification.
    1. User is unable to log into the web application.
    3. User is not granted access to the Knowledge Transfer Manager.
- Toolbox Broker fails.
    2. Request to open the Knowledge Transfer Manager fails.
    3. User is not directed to the Knowledge Transfer Manager.
    7. Upload request to the Knowledge Repository fails.
- User does not have executive permissions.
    2. Access Control System denies access to the Knowledge Transfer Manager.

Success Postcondition: User is able to add to and edit their transition documents, and those documents are saved in the Knowledge Repository.

Justification: This scenario describes the general flow of how transition documents are written, updated, and saved, directly aligning with FR-KT-1, FR-KT-2, FR-KT-3, and FR-KT-4.

S.4.4 – Performance Analysis

Use Case Name: Performance Analysis
Primary Actor: User
Precondition:

- User has an account on the Troitsky Toolbox web application with general member permissions.

Trigger: A Troitsky member would like to perform a performance analysis on a STEP file.
Main Success Scenario:

1. User logs into the web application and is successfully authorized by the Access Control System.
2. User requests access to the Performance Analysis System from the Toolbox Broker.
3. Toolbox Broker directs the user to the Performance Analysis System.
4. User inputs a STEP file, either newly uploaded or selected from the Knowledge Repository, into the Performance Analysis System.
5. Performance Analysis System calls an external API to estimate the load capacity and potential weak points of the bridge, and returns them as outputs.

Secondary Scenarios:

- Access Control System fails verification.
    1. User is unable to log into the web application.
- Toolbox Broker fails.
    2. User is not redirected to the Performance Analysis System.
    4. Because the Toolbox Broker facilitates the transfer of data from the Knowledge Repository to the Performance Analysis System, the Performance Analysis System fails to retrieve the STEP file.
- External API calls fail.
    5. Performance Analysis System fails to return the load capacity and potential weak points.

Success Postcondition: User receives an estimated load capacity and the potential weak points for their STEP file.

Justification: This is the general path that use of the Performance Analysis System will follow, and it aligns with FR-MA-1, FR-MA-4, FR-MA-5, FR-PA-1, and FR-PA-2.

S.4.5 – Rule Compliance Verification

Use Case Name: Rule Compliance Verification
Primary Actor: User
Precondition:

- User has an account on the Troitsky Toolbox web application with general member permissions.

Trigger: A Troitsky member would like to analyze their model's compliance with the competition rules.
Main Success Scenario:

1. User logs into the web application and is successfully authorized by the Access Control System.
2. User requests access to the Rule Compliance System from the Toolbox Broker.
3. Toolbox Broker directs the user to the Rule Compliance System.
4. User inputs a STEP file, either newly uploaded or selected from the Knowledge Repository, into the Rule Compliance System.
5. Rule Compliance System calls an external API to calculate the size, weight, and other metrics of the bridge, and returns which of them fall within the competition's constraints, along with the assumptions and limitations of each estimate.

Secondary Scenarios:

- Access Control System fails verification.
  1. User is unable to log into the web application.
- Toolbox Broker fails.
  2. User is not redirected to the Rule Compliance System.
  4. Because the Toolbox Broker facilitates the transfer of data from the Knowledge Repository to the Rule Compliance System, the Rule Compliance System fails to retrieve the STEP file.
- External API calls fail.
  5. Rule Compliance System fails to return the compliance results.

Success Postcondition: User learns whether their model complies with the current competition rules.

Justification: This is the general path that use of the Rule Compliance System will follow, and it aligns with FR-MA-1, FR-MA-4, FR-MA-5, FR-RC-1, and FR-RC-3.

S.4.6 – Materials Data Publication

Use Case Name: Materials Data Publication
Primary Actor: User
Precondition:

- User has an account on the Troitsky Toolbox web application with Materials Team permissions.

Trigger: A Troitsky Materials Team member would like to add or update materials data.
Main Success Scenario:

1. User logs into the web application and is successfully authorized by the Access Control System.
2. User requests access to the Material Testing Manager from the Toolbox Broker.
3. Toolbox Broker confirms with the Access Control System that the user has the authority to access the Material Testing Manager.
4. User is directed to the Material Testing Manager.
5. User inputs and saves the appropriate material property data.
6. Toolbox Broker saves the material property data to the Knowledge Repository.

Secondary Scenarios:

- Access Control System fails verification.
    1. User is unable to log into the web application.
    2. Toolbox Broker is unable to confirm the user's permissions, and cannot redirect the user to the Material Testing Manager.
- Toolbox Broker fails.
    2. User permissions cannot be confirmed, and the user is not redirected to the Material Testing Manager.
    6. Toolbox Broker cannot save the material property data to the Knowledge Repository.

Success Postcondition: Material property data is saved to the Knowledge Repository.

Justification: This scenario describes the general process the materials testing team follows to save their data, and directly corresponds with FR-MT-2.

S.5 Prioritization

This section classifies the requirements of S.2, the interfaces of S.3, the usage scenarios of S.4, and the safety and security requirements identified in the Hazard Analysis by their degree of criticality, using the MoSCoW method:

- **Must have (M):** Essential for the system to meet its goals. The Toolbox cannot be released without it.
- **Should have (S):** Important, and planned for the capstone release, but the system can still meet its core goals without it.
- **Could have (C):** Desirable, and implemented if time allows. Otherwise left for future software subteams.
- **Won't have (W):** Not planned for the current release.

The criteria used to assign each priority are explained in S.5.12, and the influence of security concerns in S.5.13. S.5.14 then uses these priorities to plan the order in which the requirements will be implemented.

S.5.1 Web Server

Table 3: Priority table for the Web Server

ID	Requirement	Priority	Reasoning
FR-WS-1	Public landing page	Should	Useful for announcements and recruitment (survey feature K), but no club service depends on it.
FR-WS-2	Login and logout	Must	No club service or data can be reached without logging in.
FR-WS-3	Request forwarding	Must	The only path from the browser to the broker, and therefore to every service.
NFR-WS-1	Serve web app	Must	The web app is the only way members use the Toolbox.
NFR-WS-2	Encrypted communication	Must	Protects passwords and unreleased team designs on shared networks such as campus Wi-Fi.
NFR-WS-3	Session expiry	Should	Reduces the risk of an unattended browser, but the system remains secure enough for an initial release without it.

S.5.2 Access Control System

Table 4: Priority table for the Access Control System

ID	Requirement	Priority	Reasoning
FR-AC-1	Authentication	Must	Limits club data to members, as required by the Problem Statement.
FR-AC-2	Roles	Must	Every permission decision in the system depends on the member's role.
FR-AC-3	Authorization	Must	The broker relies on it to check every request (FR-BR-2).
FR-AC-4	Member management	Must	Without it, accounts could not be created or removed as members join and graduate.
FR-AC-5	Team membership	Must	Required for team data isolation (FR-AC-6).
FR-AC-6	Team data isolation	Must	Teams compete with each other, so exposing a current design to a rival team would make the Toolbox unusable for current work.
NFR-AC-1	Credential security	Must	Members reuse passwords, so a database leak must not expose them.
NFR-AC-2	Effective role alterations	Should	Important for isolation when members change teams, but such changes are infrequent and a short delay is a limited risk.

S.5.3 Toolbox Broker

Table 5: Priority table for the Toolbox Broker

ID	Requirement	Priority	Reasoning
FR-BR-1	Service registration	Must	The broker cannot route requests to services it does not know about.
FR-BR-2	Request authorization	Must	The single point at which access control is enforced for every service.
FR-BR-3	Request routing	Must	All communication between the web app and the services depends on it.
FR-BR-4	Unavailable services	Should	Improves the experience when a service fails, and supports service independence (NFR-SYS-3), but is not needed for normal operation.
NFR-BR-1	Extensibility	Should	Central to long-term maintenance by future subteams, but not required for the initial set of tools to work.
NFR-BR-2	Routing overhead	Should	Affects every response time, but the club's small number of users makes a violation unlikely.
NFR-BR-3	Request logging	Should	The main diagnostic tool for future maintainers, and supports privacy investigations, but not needed for the system to function.

S.5.4 Knowledge Repository

Table 6: Priority table for the Knowledge Repository

ID	Requirement	Priority	Reasoning
FR-KR-1	Data storage	Must	Every other service stores its data here. Centralizing the club's information is the project's first goal.
FR-KR-2	Content upload	Must	The repository can only grow if members can add to it.
FR-KR-3	Search and filter	Must	Supports the archive of past designs (survey feature A), the second-highest priority feature.
FR-KR-4	View and download	Must	Stored information is only useful if members can open it.
FR-KR-5	Content management	Should	Keeps the repository organized over time, but is not needed for it to be useful in its first year.
NFR-KR-1	Search responsiveness	Should	Important for usability, but the repository will hold far fewer than MAX_REPOSITORY_ITEMS items initially.
NFR-KR-2	Data durability	Must	Once the repository replaces the club's scattered storage, it holds the only copy of much of the club's history.

S.5.5 Model Analysis System

Table 7: Priority table for the Model Analysis System

ID	Requirement	Priority	Reasoning
FR-MA-1	File validation	Must	Analysing an invalid model would produce meaningless results that members might act on.
FR-MA-2	3D visualization	Must	Rated lowest on its own (survey feature F), but needed to confirm the model was read correctly and to inspect past designs. It is also the focus of the Proof of Concept.
FR-MA-3	Data extraction	Must	Both analysis subsystems depend on the extracted data.
FR-MA-4	Current model storage	Must	Lets a team share its current design with all of its members.
FR-MA-5	Past design viewing	Must	Access to past designs was the most common frustration in the survey (feature A).
FR-MA-6	Archiving current models	Should	Needed to grow the archive each year, but only first needed at the end of the first season, and could be done manually until then.
NFR-MA-1	Model upload (STEP)	Must	No model can enter the system without it.
NFR-MA-2	Load time	Should	Affects usability, but slow loading does not prevent analysis.
NFR-MA-3	Extraction accuracy	Must	Inaccurate dimensions could let a bridge that violates a size rule pass the compliance check.

S.5.5.1 Performance Analysis System

Table 8: Priority table for the Performance Analysis System

ID	Requirement	Priority	Reasoning
FR-PA-1	Failure load estimate	Must	The highest-priority feature in the member survey (feature E, 12 top-3 picks).
FR-PA-2	Load-to-mass ratio	Must	The third-highest priority feature (feature D), and computed directly from FR-PA-1 and the extracted mass.
FR-PA-3	Material selection	Must	A failure load cannot be estimated without material properties, which come from the club's own test data.
FR-PA-4	Assumption disclosure	Must	Prevents inexperienced members from over-trusting an estimate, a safety concern identified in the Hazard Analysis.
NFR-PA-1	Estimate accuracy	Should	The goal for the estimates, but it can only be verified for past bridges whose failure loads were recorded.
NFR-PA-2	Computation time	Should	Analysis is requested occasionally, so a slower result is inconvenient rather than blocking.
NFR-PA-3	Extensibility	Could	Benefits future subteams adding new metrics, but does not affect the initial metrics.

S.5.5.2 Rule Compliance System

Table 9: Priority table for the Rule Compliance System

ID	Requirement	Priority	Reasoning
FR-RC-1	Rule import	Should	Rule compliance (survey feature J) was rated useful, but below the performance and repository features.
FR-RC-2	Rule configuration	Should	Needed for compliance checks, which are themselves a Should.
FR-RC-3	Compliance check	Should	Catches violations before construction, but teams can still check the rules by hand.
FR-RC-4	Violation details	Should	Makes a failed check actionable, but a pass or fail result alone is still useful.
FR-RC-5	Manual checks	Should	Required whenever compliance checking is provided, so that an all-pass result is not mistaken for full compliance.
NFR-RC-1	Check correctness	Must	If compliance checking is provided, a false pass could lead to a disqualified bridge. A check that gives false passes is worse than no check.
NFR-RC-2	Rule updatability	Should	Rules change every year, but the software subteam can update them in the first year if executives cannot.

S.5.6 Material Testing Manager

Table 10: Priority table for the Material Testing Manager

ID	Requirement	Priority	Reasoning
FR-MT-1	Result entry	Must	A database of material test results (survey feature B) has the highest mean rating of any feature.
FR-MT-2	Input validation	Must	A single entry error would distort every failure load estimate that uses the data.
FR-MT-3	Historical comparison	Should	Helps teams spot unusual batches and testing errors, but results remain usable without it.
FR-MT-4	Material properties	Must	The Performance Analysis System depends on it (FR-PA-3).
FR-MT-5	Result review	Should	Lets executives exclude bad results, but input validation (FR-MT-2) catches the most common errors.
NFR-MT-1	Result import	Must	The club's existing and past test data is stored in spreadsheets, and importing it is how that data enters the system.
NFR-MT-2	Traceability	Should	Lets members check the data behind a questionable estimate, but is not needed to produce estimates.
NFR-MT-3	Export	Could	Convenient for members' own analysis, but not part of any core workflow.

S.5.7 Knowledge Transfer Manager

Table 11: Priority table for the Knowledge Transfer Manager

ID	Requirement	Priority	Reasoning
FR-KT-1	Exit document templates	Must	Ease of transition between executives is a main goal identified with the Troitsky executives, and exit documents were rated highly (survey feature G).
FR-KT-2	Exit document submission	Must	Templates are only useful if members can complete and submit them.
FR-KT-3	Role records	Should	Improves on exit documents by keeping a current description of each role, but exit documents alone already preserve the key information.
FR-KT-4	Publication	Must	Exit documents must be findable by future members to preserve knowledge.
FR-KT-5	Outstanding documents	Could	Helps executives follow up, but they can track submissions informally.
NFR-KT-1	Findability	Should	Role records only reduce the support burden if new executives can find them, but search (FR-KR-3) already provides a basic way to do so.
NFR-KT-2	Longevity (PDF export)	Could	Protects against the Toolbox being retired, which is not expected in the near term.

S.5.8 System-Wide Requirements

Table 12: Priority table for the system-wide requirements

ID	Requirement	Priority	Reasoning
NFR-SYS-1	Browser support	Must	Members must be able to use the Toolbox on the computers they already have.
NFR-SYS-2	Capacity	Should	Usage peaks, such as before the competition, involve at most the club's members, so this is unlikely to be exceeded.
NFR-SYS-3	Service independence	Should	Protects the repository from failures in the more complex analysis services, but the broker architecture already isolates services.
NFR-SYS-4	Maintainability	Must	The Problem Statement identifies maintenance by a club software subteam as fundamentally crucial to the project's success.

S.5.9 Usage Scenarios

The detailed usage scenarios of S.4 are prioritized below. A scenario's priority is the priority of the requirements it depends on.

Table 13: Priority table for the usage scenarios

Scenario	Name	Priority	Reasoning
S.4.1	Knowledge Repository Information Viewing	Must	The most common use of the Toolbox, and the core of the information centralization goal.
S.4.2	Knowledge Repository Information Addition	Must	The repository only preserves knowledge if members can add to it.
S.4.3	Transition Information Addition	Must	Supports the knowledge transfer goal, which depends on exit documents being written (FR-KT-1, FR-KT-2).
S.4.4	Performance Analysis	Must	Delivers the highest-priority feature in the member survey (FR-PA-1).
S.4.5	Rule Compliance Verification	Should	Depends on the Rule Compliance System, whose functional requirements are Should.
S.4.6	Materials Data Publication	Must	Material data is required for performance analysis, and is the highest-rated feature in the survey.

S.5.10 Interfaces

The interfaces described in S.3 are prioritized below. Each interface takes the highest priority of the requirements that depend on it.

Table 14: Priority table for the interfaces

Interface	Priority	Reasoning
RESTful API between the front end and the back end	Must	Every member-facing requirement reaches the back end through this interface, including login (FR-WS-2) and request forwarding (FR-WS-3).
External static modelling libraries	Must	Data extraction (FR-MA-3) and the failure load estimate (FR-PA-1) depend on them, and both are Must.
External data server	Must	Holds all of the club's stored data, so data storage (FR-KR-1) and data durability (NFR-KR-2) depend on it.
Login aid (login service or password hashing)	Must	Required for authentication (FR-AC-1) and credential security (NFR-AC-1).

All four interfaces are Must, because each one supports at least one Must requirement. This is expected: S.3 lists only the interfaces the system cannot operate without, while optional behaviour is expressed in S.2 as Should and Could requirements.

S.5.11 Safety and Security Requirements

The safety and security requirements are specifically identified in the Hazard Analysis. The Hazard Analysis document also contains a breakdown and a roadmap of these requirements

S.5.12 Justification for Prioritization

Priorities were assigned using four criteria, in the following order:

1. **Dependency:** A requirement that other Must requirements depend on is itself a Must. For example, the Toolbox Broker's routing (FR-BR-3) and the Material Testing Manager's material properties (FR-MT-4) are Must because every service, and the failure load estimate, respectively, cannot work without them.

2. **Project goals:** Requirements that directly deliver one of the goals in the Problem Statement, being information centralization, model analysis, material testing support, knowledge transfer, and maintainability by a software subteam, were ranked above those that only improve them.

3. **Member survey:** Between features that serve the same goal, the member survey (P.7.1) was used to rank them. Failure load prediction (feature E), the archive of past designs (feature A), and the material test database (feature B) were the most highly rated, so their requirements are Must. Rule compliance (feature J) was rated lower, so the Rule Compliance System's functional requirements are Should.

4. **Schedule and risk:** The Development Plan identifies STEP model loading and visualization as the riskiest feature, and the focus of the Proof of Concept. Requirements on that path are Must, so that the riskiest work is done first, while requirements only needed after the first competition season, such as archiving current models (FR-MA-6), are Should.

Must requirements together form the minimum system that meets every project goal. Should requirements are planned for the capstone release, and Could requirements are left to future software subteams if time does not allow. No requirement in S.2 is currently classified as Won't have. Features that were deliberately left out of S.2, such as onboarding material (survey feature H) and the identification of a bridge's weak points, are not part of this release, and the extensibility requirements (NFR-BR-1, NFR-PA-3) ensure they can be added later without changing existing services.

S.5.13 Role of Security Concerns

Security and safety concerns raised the priority of several requirements that would otherwise have been ranked lower. The club's teams compete against each other, so a leak of a team's current design to another team would undermine trust in the Toolbox and make teams unwilling to store their work in it. For this reason, every requirement that enforces access control or team data isolation, being FR-AC-1 to FR-AC-6, FR-BR-2, and SR-KR-3, is Must, as are encrypted communication (NFR-WS-2) and password hashing (NFR-AC-1), which protect members' credentials.

Safety concerns affected the analysis tools in the same way. Although the Toolbox does not control any physical equipment, a team that trusts an incorrect estimate or compliance result may build, or load test, a weaker bridge, or one that is disqualified. Requirements that prevent results from being over-trusted, being assumption disclosure (FR-PA-4), error reporting (SR-PA-1), and the absence of false passes (NFR-RC-1), are therefore Must, even where the feature they belong to is not. Finally, because the Knowledge Repository becomes the only copy of the club's history, the requirements that protect it from loss, being data durability (NFR-KR-2), external backups (SR-KR-1), and error reporting on storage (SR-KR-5), are Must.

S.5.14 Phase-In Plan

The requirements will be implemented in three phases, each ending at one of the course's demonstration milestones. The dates below are taken from the capstone course calendar. Each phase builds on the previous one, so that the system is usable at the end of every phase, and the riskiest and highest-priority requirements are implemented first.

Table 15: Phase-in plan for the requirements

Phase	Milestone dates	Requirements implemented
1. Proof of Concept	PoC demonstrations, 23 November to 3 December 2026	The STEP model loading and visualization path: NFR-WS-1, NFR-MA-1, FR-MA-1, FR-MA-2, and FR-MA-3, using the external static modelling libraries (S.3).
2. Revision 0	Rev 0 presentations, 1 to 11 February 2027	All remaining Must requirements, apart from those of the Rule Compliance System: **Web Server:** FR-WS-2, FR-WS-3, NFR-WS-2 **Access Control:** FR-AC-1 to FR-AC-6, NFR-AC-1 **Broker:** FR-BR-1, FR-BR-2, FR-BR-3 **Repository:** FR-KR-1 to FR-KR-4, NFR-KR-2 **Model Analysis:** FR-MA-4, FR-MA-5, NFR-MA-3 **Performance Analysis:** FR-PA-1 to FR-PA-4 **Material Testing:** FR-MT-1, FR-MT-2, FR-MT-4, NFR-MT-1 **Knowledge Transfer:** FR-KT-1, FR-KT-2, FR-KT-4 **System-wide:** NFR-SYS-1, NFR-SYS-4
3. Revision 1 (final)	Final demonstrations, 15 to 21 March 2027; final documentation due 31 March 2027	The Rule Compliance System, including its Must requirement NFR-RC-1, and all Should requirements: **Web Server:** FR-WS-1, NFR-WS-3 **Access Control:** NFR-AC-2 **Broker:** FR-BR-4, NFR-BR-1, NFR-BR-2, NFR-BR-3 **Repository:** FR-KR-5, NFR-KR-1 **Model Analysis:** FR-MA-6, NFR-MA-2 **Performance Analysis:** NFR-PA-1, NFR-PA-2 **Rule Compliance:** FR-RC-1 to FR-RC-5, NFR-RC-1, NFR-RC-2 **Material Testing:** FR-MT-3, FR-MT-5, NFR-MT-2 **Knowledge Transfer:** FR-KT-3, NFR-KT-1 **System-wide:** NFR-SYS-2, NFR-SYS-3
After Revision 1	If time allows before the final demonstrations, otherwise left for future software subteams	The Could requirements: NFR-PA-3, NFR-MT-3, FR-KT-5, NFR-KT-2.

Phase 1: Proof of Concept. The Development Plan identifies loading a STEP model, displaying it, and extracting its data as the riskiest part of the project, so it is implemented first. Demonstrating it at the Proof of Concept confirms that the Model Analysis System, on which both analysis subsystems depend, is feasible before the rest of the system is built around it. If the demonstration shows that this path is not feasible, the Development Plan's risk mitigation applies, and the scope of the analysis tools is revised before Revision 0.

Phase 2: Revision 0. Revision 0 delivers the minimum system that meets every project goal: members can log in, store and search the club's information, analyse their team's model and estimate its failure load, record material tests, and write exit documents. The Web Server, Access Control System, Toolbox Broker, and Knowledge Repository are implemented first within this phase, since every other service depends on them (Table 20). All safety and security requirements classified as Must, such as team data isolation (FR-AC-6) and data durability (NFR-KR-2), are part of this phase, so that the system is secure from the first time members use it.

Phase 3: Revision 1. The final phase adds the Rule Compliance System, which was ranked below the other analysis tools in the member survey, together with the Should requirements that improve the usability, performance, and maintainability of the services delivered at Revision 0. Feedback from the Revision 0 demonstration and from usability testing is also addressed in this phase. Because NFR-RC-1 only applies once compliance checking exists, it is implemented together with the Rule Compliance System rather than at Revision 0.

After Revision 1. The Could requirements are implemented if time allows before the final demonstrations. Otherwise, they are documented as future work for the club's software subteam. The extensibility requirements (NFR-BR-1, NFR-PA-3) ensure that they, and features outside the current scope, can be added later without changing existing services.

The phasing of the safety and security requirements from the Hazard Analysis is given in the Hazard Analysis roadmap.

S.6 Verification and acceptance criteria

Validation overview:

To validate the system, extensive consultation with McMaster's Troitsky club will be done. Each part will be shown to the club multiple times over an iterative process to tailor the project to their unique needs. The parts demonstrated will be the role-based access system, the knowledge transfer, document upload and knowledge repository retrieval systems, the materials testing system and the model analysis system. While most components are auxiliary and

relatively simple, the materials system and model analysis systems will be difficult, the latter far more than the former, and extensive consultation and testing will be required. Early testing will use simplified models, including elementary materials data and bridge models which are far less complex than actual bridge models and materials inputs the club develops, and complexity will be iteratively increased after a lower tranche of complexity is deemed satisfactory. Key Stakeholders include the Troitsky club, from its president, the rest of its executive team, the bridge and materials developers and other members, because every member will interact with this system once it is complete.

Unit Testing:
Unit tests will be extensively performed to verify the correctness of individual systems, including

- The role-based access system;
- The knowledge repository system, namely its upload, transfer, and retrieval and download subsystems;
- The materials testing system; and
- The model analysis system.

By isolating components, errors can be isolated and caught early. These tests will point developers to areas that require fixing to make the system work correctly so the system will correctly function without a Herculean effort in the end to fix a great multiplicity of errors.

Cross-Platform Environment Compatibility:
The system shall be tested on Windows, Macintosh and Linux personal computers. Furthermore it shall be tested on the common browsers including Chrome, Edge, Firefox and Safari to ensure compatibility. This is necessary because users shall be able to access this application on the device and browser of their choice, assuming the device or browser type is sufficiently common.

Real-Time Decision Making
To validate real-time decision making abilities of the system:

- Latency profiling and timing tests shall be done
- Verifying that all tasks, especially the model analysis and materials testing, finish within the limits defined above
- Continious integration (CI) pipelines shall be implemented to ensure changes to any component, especially frequently expected ones like rule changes and role changes, do not negatively impact the response time

Data Privacy and Ethical Compliance:
While private user data is not used directly by the system, the system may

access it. Therefore, no private data will be stored without the user's consent. However, this system takes no responsibility regarding the content of data inputted to the knowledge repository, whether through uploading a document or content submitted through a knowledge transfer form.

It shall be explicitly stated that this system will store a user's full name, the roles they hold or held within the club, and the dates when they started and ended to hold each role.

Accuracy:
The accuracy of the engineering calculations calculated by external APIs is out of scope for this project. However, all external API calls and internal calculations shall be verified by a person knowledgeable within the domain. Formalizing weighting algorithms will be a big asset in verifying accuracy of these deeply technical calculations.

Scalability:
To test the system ability to scale,

- Performance testing such as:
  - Load testing
  - Stress testing
- Verify the system can handle many different types of inputs
- Distributive testing, ensuring each member of the club can concurrently use the system without system failure

S.7 Formalization

S.7.1 FEA Payload Schema

This schema defines the structured mathematical model generated from the parsed `.stp` file, which is sent to the external FEA API.

FEAPayload
$nodes : \mathbb{P}\ NODE$ $elements : \mathbb{P}\ ELEMENT$ $elementMaterial : ELEMENT \to MATERIAL$ $materialProperties : MATERIAL \to (YoungsModulus \times YieldStrength)$ $boundaryConditions : \mathbb{P}\ NODE$ $appliedLoad : \mathbb{R}$
$\forall e \in elements \bullet elementMaterial(e) \in \text{dom}\ materialProperties$ $boundaryConditions \subseteq nodes$ $appliedLoad > 0$

Where:

- $\mathbb{P}$ represents the power set (a set of sets), meaning *nodes* and *elements* are collections of multiple structural pieces.
- $\rightarrow$ represents a total function, ensuring that *every* extracted element is mapped to exactly one material.
- $\times$ represents the Cartesian product, meaning material properties consist of a tuple (e.g., Young's Modulus and Yield Strength).
- dom refers to the domain of the function, ensuring the material assigned to an element actually exists in our material properties database.
- The predicate section (below the middle line) enforces the strict constraints required by the API: all materials must be known, boundary conditions must be valid nodes, and the load must be positive.

S.7.2 Mathematical Definition of Predictive Analysis (Post-Processing)

While the external Finite Element Analysis (FEA) API computes the complex differential equations required to simulate load distribution, the Troitsky Toolbox is responsible for the predictive logic. This section mathematically formalizes how the system bridges the gap between raw FEA output and the user-facing bridge failure prediction.

Let E be the set of all structural elements (e.g., popsicle sticks) in the bridge model. When the FEA API returns its analysis, let $\sigma_{API}(e)$ represent the maximum axial stress computed for element $e \in E$.

Let $\mu_{strength}(m)$ be the average ultimate strength of a material m, calculated from the club's historical material testing repository.

The system determines the **Factor of Safety (FoS)** for each individual element e as the ratio of the material's historical strength to the stress applied under the simulated load:

$$FoS(e) = \frac{\mu_{strength}(elementMaterial(e))}{\sigma_{API}(e)} \tag{1}$$

The system predicts that the bridge will **fail** under the applied load if there exists at least one element where the applied stress exceeds the material's ultimate strength (i.e., the Factor of Safety drops below 1). This is formalized logically as:

$$\text{BridgeFailure} \iff \exists e \in E \mid FoS(e) < 1 \tag{2}$$

By utilizing this formal logic, the system isolates the custom predictive features (based on Troitsky's unique materials) from the standard physics simulation provided by the external API.

S.7.3 Z-Notation Formalization of Rule Compliance

To fulfill the goal of automating rule compliance checks for the Concordia Troitsky competition, the physical constraints of the bridge must be formally defined.

First, we define the state schema of a physical bridge extracted from the `.stp` CAD file:

BridgeModel
$totalMass : \mathbb{R}$
$length : \mathbb{R}$
$width : \mathbb{R}$
$clearanceHeight : \mathbb{R}$
$totalMass > 0$
$length > 0 \land width > 0 \land clearanceHeight > 0$

Next, we define the rule compliance operation. This schema takes a `BridgeModel` and evaluates it against the competition's geometric and mass constraints, outputting a Boolean value (`isValid!`) to the user interface.

RuleComplianceCheck
$\Xi BridgeModel$
$MaxAllowedMass, MinRequiredLength, MaxAllowedLength : \mathbb{R}$
$MinRequiredWidth, MaxAllowedWidth, MinClearance : \mathbb{R}$
$isValid! : \mathbb{B}$
$isValid! = (totalMass \leq MaxAllowedMass) \land$ $(length \geq MinRequiredLength \land length \leq MaxAllowedLength) \land$ $(width \geq MinRequiredWidth \land width \leq MaxAllowedWidth) \land$ $(clearanceHeight \geq MinClearance)$

Where:

- $\Xi BridgeModel$ indicates that this operation observes the state of the `BridgeModel` but does not alter or mutate the bridge data itself.
- $\mathbb{B}$ represents a Boolean value (True/False) returned to the frontend.
- $\land$ represents the logical AND operator, ensuring that a bridge is only deemed valid if *all* structural and mass constraints are simultaneously satisfied.

S.7 Formalizations

The nature of the analysis is to be determined, so this formalization shall be updated in the future. However, the weighting can be formalized now, so a model will be shown below.

Let $r_1, r_2, ..., r_n$ represent hard constraints 1 to n. Let $w_1, w_2, ..., w_m$ represent the weights for objectives 1 to m. Let $s_1, s_2, ..., s_m$ represent the scores for each objective 1 to m. Let $b_1, b_2, ..., b_k$ represent whether a soft constraint has been violated. There are k soft constraints. Let p_i represent the penalty for violating soft constraint i.

So the weighted score W is

$$W = \prod_{i=1}^{n} r_i (\sum_{i=1}^{m} w_i s_i - \sum_{i=1}^{k} b_i p_i)$$

P Project

P.1 Roles and personnel

P.1.1 Project Supervisor - Mason

The project supervisor will be responsible for coordinating internal deadlines, meetings and development timelines. The supervisor will serve as the main point of contact with administration, the supervisor and the TA, and serve as the default team lead.

P.1.2 Troitsky Club Communication Associate - Pedro

The Troitsky Club Communication Associate will be responsible for the majority of the coordination with the club organization and executive members. Surveys and interviews will be organized, distributed, and run by this member.

P.1.3 Front-end / UI Developer - Pedro, Michael

This role will be responsible for the UI of the product, both in technical displays of data and in more theme/design heavy components. They will build the web app's interface, so experience in web frameworks would be beneficial. They will be the ones primarily responsible for the usability testing in the Winter term's CD.

P.1.4 Back-end/Data Developer - Jack, Mason

This role will be responsible for the design of the data storage within the app. This will cover both the files and data the club stores (bridge files, tutorials), but also RBAC information and user/club site preferences.

P.1.5 Quality Assurance / Tester - Michael

The Quality Assurance / Tester will be responsible for maintaining a standard of quality in both code and documentation of the web app. This will involve a review of final product content and the overall design of the testing within the system.

P.1.6 Supervisor - Isabella Vescio

The supervisor for the project will provide guidance, domain expertise and general feedback for the service. As they are heavily involved in the club, they will also support interaction and integration with the club.

P.1.7 Teaching Assistant - Melvin Mokhtari

The TA will provide course-related feedback, answer administrative questions and provide feedback on expectations and course grading. They will serve as the main point of contact with course administration. General project feedback during TA meetings will also help keep the project timeline in check.

P.2 Imposed technical choices

- **Platform/delivery:** Web app for desktop browsers. Accessible through traditional browsers such as Google Chrome, Firefox, Edge, and Safari. Requires a working internet connection to utilize the service.
  Rationale: Ensures that the service is accessible by all club members. Also allows the Troitsky toolbox to act as a landing page for the McMaster club.

- **File Formats:** STEP (.stp/.step) files for loading and submitting models. CSV and Excel data formats (.csv/.xlsx) for importing material testing results.
  Rationale: Standardized format among possible external modelling software that teams currently use. Ensures compatibility with current and future model files (E.4). CSV and Excel data files are both tabular and work with current material testing data (E.2).

- **Back-end Programming language:** C# on .NET.
  Rationale: LINQ availability and ease of use, and the ease of maintainability by future software subteam members.

- **Testing, Linting and Coding Standard:** xUnit/bUnit + Coverlet for coverage, `dotnet format`, Standard .NET coding conventions
  Rationale: xUnit and Coverlet are widely documented; bUnit, running on top of xUnit, supports the testing of Blazor components. `dotnet format` ships with the .NET SDK and can work with config files to integrate with the .NET compiler.

- **Authentication (RBAC):** ASP.NET Core Identity.
  Rationale: RBAC matches the club structure, and Identity is built into the ASP.NET Core stack.

P.3 Schedule and milestones

Table 16: Schedule

Submission Items	**Date**
• Problem Statement • Proof of Concept Plan • Development Plan	28/09/2026
• Problem Statement Peer Review • Proof of Concept Plan Peer Review • Development Plan Peer Review	01/10/2026
• SRS • Hazard Analysis	19/10/2026
SRS Peer Review	22/10/2026
• VnV Peer Review Requests • POC Requests	05/11/2026
• Design Doc Rev 1 • Fall Custom Deliverable	19/11/2026
Module Implementation	21/01/2027
Exposition Registration	27/01/2027
Final Demo Schedule Requests	01/03/2027
• VnV • Custom Deliverable Winter	11/03/2027
Team Contribution Survey	18/03/2027
Final Submission	31/03/2027
Expo	06/04/2027

P.4 Tasks and deliverables

Table 17: Tasks and Deliverables

Task	Description	Deliverable	Date
Requirements Peer Review	Peer reviewing another group's Software Requirements Specification.	SRS Peer Review	22/10/2026
Verification, Validation & POC Prep	Plan for system testing and request peer reviews for the Proof of Concept.	V&V Plan Revision 0, POC Requests	05/11/2026
Initial Design & Fall Custom Document	Create high-level system design and execute stakeholder interviews to gather domain knowledge.	Design Doc Rev -1, CD-Fall (Interviews)	19/11/2026
Core Implementation & Design Iteration	Implementation of core modules (e.g., repository, STEP file parsing) and detailed design documentation.	Module Implementation (Design Doc Rev 0)	21/01/2027
Expo Preparation	Administrative task to register the team for the final year-end EXPO event.	Exposition Registration	27/01/2027
Final Demo Planning	Coordinate and request scheduling for the final system demonstration.	Final Demo Schedule Requests	01/03/2027
System Testing & Winter Custom Document	Execute usability testing of the toolbox and finalize verification & validation reports.	V&V Report Rev 0, CD-Winter (Usability Testing)	11/03/2027
Team Evaluation	Complete surveys to determine individual team contribution factors (TCF).	Team Contribution Survey	18/03/2027

Task	Description	Deliverable	Date
Final Documentation	Finalize all project documentation, fix final bugs, and submit source code.	Final Submission (Revision 1 of all docs)	31/03/2027
Project Presentation	Showcase the completed Troitsky Toolbox to the department, public, and stakeholders.	EXPO Demonstration	06/04/2027

P.5 Required technology elements

- **Database server:** Microsoft SQL Server.
  Rationale: Used to store the shared repository and uploaded models. Pairs with C# and LINQ. Azure for students can be used to access SQL Server Development Edition free (testing only).

- **Hosting/Deployment:** Azure App Service; if the university server is accessible, that would be preferred. Docker will be used to containerize the service in app development and deployment.
  Rationale: Azure for students also provides limited access to hosting, which is why the university servers are preferred. Docker allows for standardization between development and deployment.

- **Version control and CI:** A GitHub repository with GitHub Actions.
  Rationale: Given by course. Runs with xUnit/Coverlet and `dotnet format`.

- **Structural Analysis Library:** BriefFiniteElement.NET (BFE.NET).
  Rationale: Open-source (LGPL-3.0) linear-static finite element library written in C#, supporting 3D frame and truss models. Fallback: a direct stiffness solver written by the team using Math.NET Numerics (MIT).

- **STEP Parsing Library:** IxMilia.Step.
  Rationale: Open-source (MIT) STEP reader written in C#, used to extract member geometry on the server. As it supports a limited set of STEP entities, it will be tested on club STEP exports in the Proof of Concept. Fallback: Open CASCADE (LGPL-2.1) through a C++/CLI wrapper, which requires Windows hosting.

- **3D Model Viewer:** occt-import-js with three.js in the browser.
  Rationale: The web app cannot render 3D models in C#. occt-import-js (Open CASCADE compiled to WebAssembly, LGPL-2.1) reads STEP files in the browser and passes the mesh to three.js for display.

P.6 Risks and mitigation analysis

P.6.1 Risk Analysis

- **Risk: 3D Model Import and Analysis is Infeasible**
  - **Description:** Parsing STEP (.stp) files, rendering them, and extracting member geometry for the rule compliance and performance prediction tools is the most technically complex feature of the Toolbox. Available libraries may not support the required data extraction, or may not run in a web environment.
  - **Mitigation:** Prove the import and visualization pipeline in the Proof of Concept demonstration before dependent tools are built. Use an established geometry library rather than a custom parser, and agree on a standard CAD export template with the club to limit the variety of input files.
  - **Contingency:** Fall back to manual or spreadsheet entry of member geometry so the rule compliance and prediction tools remain usable without 3D import.
- **Risk: Insufficient Legacy Club Data**
  - **Description:** Past designs, material testing results and documentation are scattered across members and inconsistent in format, so the information repository may launch with little useful content.
  - **Mitigation:** Collect existing documents with the club executives early in the project and define standard upload templates for designs and material tests.
  - **Contingency:** Launch with the current season's data only, and plan for the repository to be populated over time through the upload tools.
- **Risk: Technical Issues with External Libraries**
  - **Description:** Technical issues may arise with external technologies used by tools such as bridge modelling and rule compliance, including licensing or deployment limits (for example, Autodesk APIs requiring a desktop installation).
  - **Mitigation:** Confirm licensing and deployment constraints of each library before adopting it. Team members test and validate their code before opening a pull request, and at least one other member reviews it.
  - **Contingency:** Replace the library with an open-source alternative, or reduce the affected tool's scope.
- **Risk: Unrealistic Schedule or Workload**
  - **Description:** Tasks may take longer than expected or workload may become unbalanced among team members.

- **Mitigation:** Track tasks and milestones on the GitHub Kanban board. The Notetaker records timelines at check-in meetings, and the team revises estimates and reassigns tasks as needed.
- **Contingency:** Because tools are largely independent, defer lower-priority tools to the software subteam's backlog rather than delaying core features.

- **Risk: User Interface is Not User-Friendly**
  - **Description:** The interface may not be intuitive for Troitsky members with varying levels of technical and modelling experience.
  - **Mitigation:** Run usability testing sessions with both new and experienced members (see the Usability Testing CD), gather feedback, and make iterative improvements.
  - **Contingency:** Prioritize fixes to the most-used workflows and add in-app guidance for complex tools.

P.6.2 Threat Analysis

- **Threat: Unauthorized access to the repository or upload tools**
  - **Description:** A user that does not have authorized access is able to upload a file or the current repository data.
  - **Mitigation:** Every member must log in, with each request being checked against the member's role and team before it is routed and their team's models/information become visible. To limit direct external user access, idle sessions timeout, making the user account unavailable, and executives deactivate the accounts of members who leave the club.
  - **Contingency:** Utilizing the broker logs to track activity, and utilizing backups to return to a secure state.

- **Threat: Malicious, corrupted or falsified file uploads (STEP models and test data)**
  - **Description:** A member uploads a corrupted, falsified or malicious file, either by mistake or on purpose. A malformed STEP model could crash or stall the Model Analysis System. False or incorrect material test results would also taint the aggregated materials property results, skewing model results from teams that reference/utilize those results.
  - **Mitigation:** Only authenticated members can upload files, given their role and current project stage. Bridge models must be valid STEP files, and invalid files are rejected. Material test results must follow the standard CSV/Excel template and pass validation for missing or physically impossible values, with metadata filled out to club standard. Uploading user is recorded.

    - **Contingency:** Identify the user and data that was uploaded through logs and upload data. Executives can archive or exclude individual files from aggregated data results.

- **Threat: Loss of club records**
    - **Description:** Club records stored in the Knowledge Repository could be lost, either maliciously or unintentionally. Loss could come from a failure of the hosting and data storage platform (E.2), a faulty update, or a member overwriting or removing content by mistake.
    - **Mitigation:** The repository is backed up at least every BACKUP_INTERVAL (7 days). Items are archived rather than deleted, and the failure of one service does not affect the others.
    - **Contingency:** Restore lost data from the most recent backup, losing at most BACKUP_INTERVAL (7 days) of contributions. Test results and handoff documents can also be exported outside the system.

P.7 Requirements process and report

P.7.1 Requirements Survey Report

McMaster Troitsky Members

A survey was sent to Troitsky members through the club's communication channels to elicit the overall wants and needs of members. The questions, as administered, are listed below and in the survey design record, and the responses are available as raw data.

1. How many years of experience do you have in Troitsky? [**0–5**]
2. What is your current role(s)? [**Team/General Member, Team Captain, Coach, Executive Member**]
3. What is your modelling experience? [**None, Through some courses, Experience modelling in and out of class**]
4. How valuable would you find an entry-level modelling service, implementing simple load testing and weak point analysis? [**1–5**]
5. Rate (1–5) how valuable you would find having this information easily accessible, regarding past McMaster Troitsky data. [**Bridge modelling file, Joint construction data (specific model use), Manufacturing information, Past team results, Past technical presentation material**]
6. How does your team store data? [**Microsoft Teams, Google Drive, Individual people hold files, Discord, Email, Other**]

7. What has been one of the most frustrating parts of your time in the club, outside of actually building the bridge? [**Paragraph**]
8. Rate these features on how useful you would find them. [**Would not use, Slightly useful, Useful, Very useful, Club needs this**]
    A. Searchable archive of past bridge designs
    B. Database of material testing results (stick strength, glue, floss, etc.)
    C. Bridge weight estimator from a submitted model
    D. Structural efficiency calculation
    E. Ultimate load and failure point prediction
    F. Upload and visualise a team's bridge model in the browser
    G. Exit documents / role handover records for executive positions
    H. Onboarding guide and training material for new members
    I. Private workspace for each team's own files and data
    J. Competition rules and regulations reference
    K. Club announcements and general information page
9. If we could deliver only 3 of these, which would you choose? [**A–K, up to 3**]
10. Would you want your team files, results and progress to be available for future teams to learn from? [**Yes, No, Maybe**]
11. If you could go back to being a new member, or if you're currently a new member, what tutorials would you want access to? [**Modelling software, Manufacturing, Slab building, Team organization, Other; select 2**]

Survey Results

Respondents

- **Number of responses** n = 20.
- **Collection Timeline:** Collected responses between October 7th and 15th.
- **Roles:** 10 general members, 6 captains, 5 executives, 2 coaches, 1 alumnus.
- **Experience:** 13 of 20 have 2 years or less in the club. All 20 have some CAD experience; 9 only through courses.

Feature Ratings and Priorities Table 18 summarizes the ratings and top-3 choices for each feature, with the requirements that address it.

Table 18: Member survey: feature ratings and priorities ($n = 20$)

Feature	Mean (1–5)	High[a]	Top-3[b]	Requirements
E. Failure load prediction	4.35	18	12	FR-PA-1
A. Archive of past designs	4.35	17	11	FR-KR-3, FR-MA-5, FR-MA-6
D. Structural efficiency	4.25	17	10	FR-PA-2
C. Weight estimator	4.10	15	8	FR-MA-3
B. Material test database	4.45	18	6	S.2.6
J. Rules reference	4.00	14	5	S.2.5.2
H. Onboarding material	4.00	14	3	—
G. Exit / handover documents	4.00	15	2	S.2.7
I. Private team workspace	3.55	11	2	FR-AC-6, FR-MA-4
K. Announcements page	3.85	14	1	FR-WS-1
F. 3D model viewer	3.55	12	0	FR-MA-2

[a] Respondents rating the feature "Very useful" or "Club needs this". Mean maps the scale "Would not use" to "Club needs this" onto 1–5.
[b] Respondents choosing the feature among the three they would keep if only three could be delivered.

Other Findings

- **Knowledge Repository Data Prioritization:** When considering what data from past teams and competition years current members wish to have, joint design and construction data was the most prioritized.
- **Current Team Data Storage Solutions:** Most teams (13/20) utilize Microsoft Teams, with the rest spread between other common services.
- **Data Sharing Preference:** 17/20 respondents have stated that they wish to have results, designs and work preserved and shared within the app.
- **Desired Tutorial Types:** For tutorials kept within the knowledge repository, there was a clear split, with more experienced members stating that they wish to have manufacturing-focused tutorials, while newer, less experienced members pushed for modelling software tutorials. Results: Manufacturing 15, modelling software 9, slab building 9, team organization 4; 1 person wrote in joint method selection

- **Major General Frustrations:** Access to past designs, reference work for guidance and orientation for new members, inspiration for designs, specific joint construction and design information.

Use of Results Primarily utilized during the requirement specification and modification through the SRS development (S1 & S2). The user base's evaluation of service value helped to determine the prioritization (S.5) for the project. User characteristics were also determined and expanded upon from the survey, especially considering the experience of the current members within the club. The survey also outlines the gaps in the srs, and past misconceptions the team had about the user base.

P.7.2 Ongoing and Future Processes

McMaster Troitsky Executives

Both Pedro Azevedo and our supervisor Bella Vescio are Troitsky Executives; our requirements are gathered through team meetings and supervisor meetings. The consolidation of information and ease of transition is one of the main goals in the development of this project, so we would want to hear the executives' opinions on this. Important Questions to Ask:

- What information is most important to consolidate?
- When taking your position, what would you have liked to have seen within a transition document?
- How would you go about outlining the duties of a software development and support team?

McMaster CAS Students

McMaster CAS students are needed for the Software Development Team; they will maintain and expand the toolbox. The challenge of this project is the maintainability and expandability of the code, so we must have an understanding of how future software teams are set up so that we can prepare documentation accordingly. To ease the burden on them, we must elicit preferred documentation formats. Some questions we would like to ask are:

- When working with a system you did not develop yourself, rate(1-5) how helpful you find these documentation formats.
  - Markdown or other lightweight markup languages for README files.
  - Inline Documentation (Javadoc, Doxygen, XML Doc Comments, JSDoc, Docstrings)
  - Collaborative Wiki Formats, think online library documentation.
  - Modelling and Visual Formats (PlantUML, C4 Model)

A Appendix

A.1 Symbolic Constants

The non-functional requirements in S.2 refer to the symbolic constants listed in Table 19, rather than to fixed values, so that a value can be revised in one place as the project progresses. Each constant is given with its current value and the rationale or reference for that value.

Each value comes from one of three sources. Where a published guideline applies, such as human response-time limits or session security practice, the value is taken from that guideline. Values marked *Troitsky discussion* were set in discussion with the Troitsky executives involved in the project, Pedro Azevedo and our supervisor Bella Vescio (P.7.2), who know how the club's teams work and what they would accept. These values are also checked against the member survey results (P.7.1) where the survey gives relevant data. Values that depend on the performance of the implementation are marked *to be refined*, and will be revised once the Proof of Concept and usability testing provide measured data.

Table 19: Symbolic constants used in the non-functional requirements

Constant	**Value**	**Used in**	**Rationale / Reference**
MAX_BROKER_-OVERHEAD	100 ms	NFR-BR-2	About 0.1 s is the limit for a response to feel instantaneous, and about 1 s is the limit for keeping a user's flow of thought uninterrupted [**?**, ch. 5]. Every request passes through the broker, so its overhead is held to the 0.1 s limit. Routing alone is then never noticeable, and most of the 1 s budget is left for the service doing the actual work.

Constant	Value	Used in	Rationale / Reference
SESSION_TIMEOUT	30 minutes	NFR-WS-3	The OWASP Session Management Cheat Sheet [**?**] gives 15–30 minute idle timeouts as common for low-risk applications. The Toolbox holds no financial or sensitive personal data, but it does hold unreleased team designs (FR-AC-6). *Troitsky discussion:* the upper end of the range was chosen because members often keep the Toolbox open beside CAD software or while reading a document, and a shorter timeout would log them out in the middle of their work.
MAX_SEARCH_TIME	2 seconds	NFR-KR-1	Delays between 1 s and 10 s are noticed but keep the user's attention on the task [**?**, ch. 5]. Members usually refine a search several times, for example by adding year, team, and tag filters, so each search is held near the low end of that range, keeping refinement quick. Search supports the archive of past designs, the second highest-priority feature in the member survey (P.7.1, feature A: 11 of 20 top-3 picks).

Constant	Value	Used in	Rationale / Reference
MAX_REPOSITORY_ITEMS	10,000 items	NFR-KR-1	*Troitsky discussion:* the club registers 4–6 teams per year (Problem Statement [**?**]; G.3). Each team produces bridge models, material test records, documents, and presentation material, and executives add exit documents and role records, which together make several hundred items per year at most. 10,000 items therefore covers more than a decade of use without the repository having to be redesigned.
BACKUP_INTERVAL	7 days	NFR-KR-2	*Troitsky discussion:* limits the worst-case loss to one week of contributions. Within a week, the members who uploaded the lost items still have their own copies: the member survey shows that teams currently keep files on Microsoft Teams, Google Drive, and individual members' devices (P.7.1). They can therefore re-upload the lost items, while a weekly backup stays simple and inexpensive to run on student hosting (P.5).

Constant	Value	Used in	Rationale / Reference
MAX_MODEL_LOAD_TIME	30 seconds	NFR-MA-2	*Troitsky discussion, to be refined:* STEP parsing and rendering are computationally expensive. Uploading a model is an occasional action, taken once per design revision, not a repeated interactive one, so the executives judged a wait of up to 30 seconds acceptable. This exceeds the 10 s limit for holding attention [**?**, ch. 5], so progress must be shown while loading. The value will be refined using Proof of Concept measurements on real club models.
DIMENSION_TOLERANCE	0.5 mm	NFR-MA-3	*Troitsky discussion:* a standard popsicle stick is about 2 mm thick [**?**, Sec. 7.1], and hand cutting and gluing produce errors of around a millimetre. A tolerance of 0.5 mm, a quarter of a stick's thickness, is finer than the precision the bridge is built to, so extraction error does not change a rule compliance result (S.2.5.2) or a mass estimate in any meaningful way. Joint tolerances were also raised as a frustration in the member survey (P.7.1).

Constant	Value	Used in	Rationale / Reference
FAILURE_LOAD_-TOLERANCE	25%	NFR-PA-1	*Troitsky discussion, to be refined:* hand-built popsicle stick bridges vary greatly in glue coverage, stick quality, and joint fit, so two copies of the same design can fail at noticeably different loads. Tighter predictions are not realistic for an initial model. The competition also scores load predictions by their relative error from the ultimate load [**?**, Sec. 10.7]. Executives agreed that an estimate within 25% is still enough to compare design alternatives and to rule out weak designs before building. Members currently have no estimate at all ("we don't know what works until we build it and it fails at competition", P.7.1), and failure load prediction is the highest-ranked feature in the survey (P.7.1, feature E). The value will be revisited once the model is validated against recorded results.
MAX_ANALYSIS_-TIME	30 seconds	NFR-PA-2	*Troitsky discussion, to be refined:* like model loading, analysis is requested occasionally, once per design revision, and not interactively. A wait of the same length as model loading, with a progress indicator, is therefore acceptable. The value will be refined using Proof of Concept measurements of the finite element solver (P.5).

Constant	Value	Used in	Rationale / Reference
MAX_ROLE_-SEARCH_TIME	2 minutes	NFR-KT-1	*Troitsky discussion:* a new executive should be able to find their role record without help, faster than it would take to message a previous executive and wait for a reply. 2 minutes allows time to log in, search or browse to the record, and open it, without prior training.
MIN_CONCUR-RENT_USERS	50 members	NFR-SYS-2	Club size: 4–6 registered teams (Problem Statement [**?**]; G.3) of 6–8 students each [**?**, Sec. 5.1], giving 24–48 competitors, plus executives and general members. Use is expected to peak just before design deadlines, when all teams may be using the Toolbox at the same time, so the system is sized for the active membership, not for average use.
MAX_ONBOARD-ING_TIME	1 week	NFR-SYS-4	*Troitsky discussion:* software subteam members are students who work on the Toolbox part-time alongside their courses, and membership changes every year (P.7.2). Allowing 1 week for setup and a first change lets a new member contribute early in the term, and makes the documentation, not the help of a previous member, the means of handover.

A.2 Traceability Matrix

Table 20 lists each requirement in S.2 with its relationships to other requirements. Each ID in the table links to the requirement in S.2, and the ID in S.2 links back to its row here. Three relationships are used:

- **Depends on:** The requirement cannot be met unless the listed require-

ments are also met.

- **Fulfils:** Meeting the requirement contributes to meeting the listed requirements.
- **Constrains:** The requirement, usually non-functional, places a limit on how the listed requirements are met.

Each relationship is listed once, in the row of the requirement it describes. For example, FR-BR-1 is listed as fulfilling NFR-BR-1, so the relationship is not repeated in the row of NFR-BR-1. Dependencies on the Toolbox Broker's routing are not listed for every service, since all communication between services passes through the broker (S.1).

Table 20: Requirements traceability matrix

Requirement	**Depends on / Fulfils / Constrains**
FR-WS-1	None
FR-WS-2	Depends on FR-AC-1
FR-WS-3	Depends on FR-AC-3, FR-BR-3
NFR-WS-1	None
NFR-WS-2	Fulfils NFR-AC-1, FR-AC-6
NFR-WS-3	Depends on FR-WS-2
FR-AC-1	Depends on FR-AC-4
FR-AC-2	Depends on FR-AC-4
FR-AC-3	Depends on FR-AC-2, FR-AC-5
FR-AC-4	None
FR-AC-5	Fulfils FR-AC-6
FR-AC-6	Depends on FR-AC-3, FR-AC-5, FR-MA-6
NFR-AC-1	Constrains FR-AC-1
NFR-AC-2	Constrains FR-AC-4, FR-AC-5 Fulfils FR-AC-6
FR-BR-1	Fulfils NFR-BR-1
FR-BR-2	Depends on FR-AC-3
FR-BR-3	Depends on FR-BR-1, FR-BR-2
FR-BR-4	Depends on FR-BR-1 Fulfils NFR-SYS-3
NFR-BR-1	Fulfils NFR-SYS-4
NFR-BR-2	Constrains FR-BR-3
NFR-BR-3	Depends on FR-BR-3 Fulfils NFR-SYS-4
FR-KR-1	None
FR-KR-2	Depends on FR-KR-1 Fulfils FR-KR-3
FR-KR-3	Depends on FR-KR-1

Requirement	Depends on / Fulfils / Constrains
FR-KR-4	Depends on FR-KR-1
FR-KR-5	Depends on FR-KR-1, FR-AC-3
NFR-KR-1	Constrains FR-KR-3
NFR-KR-2	Constrains FR-KR-1
FR-MA-1	Fulfils FR-MA-3
FR-MA-2	Depends on FR-MA-1
	Fulfils FR-MA-5
FR-MA-3	Depends on FR-MA-1
FR-MA-4	Depends on FR-AC-6, FR-KR-1, FR-MA-3
FR-MA-5	Depends on FR-MA-2, FR-MA-6
FR-MA-6	Depends on FR-MA-4
NFR-MA-1	Constrains FR-MA-1, FR-MA-4
NFR-MA-2	Constrains FR-MA-2
NFR-MA-3	Constrains FR-MA-3
	Fulfils NFR-RC-1
FR-PA-1	Depends on FR-MA-3, FR-PA-3
FR-PA-2	Depends on FR-PA-1, FR-MA-3
FR-PA-3	Depends on FR-MT-4
FR-PA-4	Depends on FR-PA-1, NFR-PA-1
NFR-PA-1	Constrains FR-PA-1
	Depends on FR-MA-5
NFR-PA-2	Constrains FR-PA-1, FR-PA-2
NFR-PA-3	Fulfils NFR-SYS-4
FR-RC-1	Depends on FR-KR-1
FR-RC-2	Fulfils NFR-RC-2
FR-RC-3	Depends on FR-MA-3, FR-RC-1
FR-RC-4	Depends on FR-RC-3
FR-RC-5	Depends on FR-RC-3
NFR-RC-1	Constrains FR-RC-3
	Depends on NFR-MA-3
NFR-RC-2	Constrains FR-RC-1, FR-RC-2
FR-MT-1	None
FR-MT-2	Fulfils FR-MT-4, FR-PA-3
FR-MT-3	Depends on FR-MT-1
FR-MT-4	Depends on FR-MT-1, FR-MT-2
FR-MT-5	Fulfils FR-MT-4, NFR-MT-2
NFR-MT-1	Depends on FR-MT-2
NFR-MT-2	Constrains FR-MT-4
NFR-MT-3	Depends on FR-MT-1
FR-KT-1	None
FR-KT-2	Depends on FR-KT-1
FR-KT-3	Depends on FR-AC-2

Requirement	Depends on / Fulfils / Constrains
FR-KT-4	Depends on FR-KT-2, FR-KT-3, FR-KR-1
FR-KT-5	Depends on FR-KT-2, FR-AC-2
NFR-KT-1	Constrains FR-KT-3 Depends on FR-KR-3
NFR-KT-2	Constrains FR-KT-2, FR-KT-3
NFR-SYS-1	Constrains NFR-WS-1
NFR-SYS-2	Constrains NFR-BR-2, NFR-KR-1, NFR-MA-2, NFR-PA-2
NFR-SYS-3	Depends on FR-BR-3
NFR-SYS-4	None

A.3 Changes from Template

Table 21 lists the changes made to the structure of the Meyer template [**?**] and the reason for each.

Table 21: Changes from the Meyer template

Change	Reason
Moved E.1 Glossary to the front of the document, before the Goals book.	The course rubric requires terms and acronyms to be defined before their first use. At the front, the glossary comes before every book that uses its terms.
Added the Appendix book (A) after the Project book.	The Meyer template has only the Goals, Environment, System and Project books. The course rubric and SRS checklist require content that belongs in none of them (A.1, A.2, A.4, A.5).
Added a References section after the Project book.	The Meyer template has no references section. It lists the sources cited in the document, such as those for the values in A.1.
Added A.1 Symbolic Constants.	The SRS checklist requires the values of symbolic constants to be given in a table in the Appendix.
Added A.2 Traceability Matrix.	The course rubric requires a table that shows the traceability between requirements.
Added A.3 Changes from Template.	The SRS checklist requires changes made to the template to be described in the document.
Added A.4 Generative AI Use Disclosure.	Required by the course rubric.
Added A.5 Reflection.	Required by the course rubric, using the questions from the course's SRS reflection template.

A.4 Reflection

Team

1. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?

2. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.

3. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or

mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Individual

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.

A.4.1 Team

1. The majority of our requirements came from our clients and their proxies; we are developing a service for a small club full of engineering students in alternate disciplines to ours, we do not have the background to write the requirements needed for this project without asking those more closely involved in the club. Additionally the size of the club makes it very realistic to at least survey the majority of the members and get a strong idea of what is needed before writing out requirements. In particular I think the functional requirements are more closely related to our conversations with our stakeholders, while the more technical requirements within the non-functional requirements we had the technical background to cover the best.
2. Below is a list of knowledge and skills that each teammate will need to learn during the corse ofthis capstone
    - **Mason Azzopardi:** Will need to learn how to work with third party dependencies in order to preform FEA on STEP file in C#.
    - **Jack Easton:** Will need to get familiar with C# as he only has a bit of experience working with it in his course work.
    - **Michael Roberts:** Will need to get familiar with front end integration using JavaScript and Razor.
    - **Pedro Azevedo:** Will need to learn to integrate the project within the day to day of the Troitsky club.
3. Below is how each group member will work on their skills/ knowledge.
    - **Mason Azzopardi:** Will read through documentation or use Gen AI to explain the dependencies to him. Most likely will read documentation first as it is more reliable.
    - **Jack Easton:** Will watch youtube tutorials or go through a free online C# course, most likely will go through youtube tutorials as it fits his learning style more.

- **Micheal Roberts:** Will use his C# experience to practice with Razor or work with Pedro who has experience with Razor. Most likely the first due to Pedros availability.
- **Pedro Azevedo:** Pedro will work on the integration of project within the club through interactions with peers and conversations with execs. He will prioritize his interactions with peers because their adoption will drive the executives adoption.

A.4.2 Pedro Azevedo

1. In writing this deliverable, the utilization of GitHub features, such as kanbans, issues, comments, PRs, milestones, tags and even kanban custom views, facilitated development and allocation of work among the team. As the SRS is such a large document, utilizing these features allowed confidence in limited conflicts appearing. The utilization of a LaTeX template with component text documents that are loaded in allowed branches to isolate their features to single documents.

2. The primary pain points came from determining what is expected within this deliverable and working with that in our project domain. As many of the examples given for the SRS are varied, there was a lack of clarity in the necessary depth for certain sections. It was also difficult working around the hyperlinking and interconnection within the document, as updates to sections would have to be reflected in several components.

3. The primary course that has helped so far is 3RA3, as that covered the development of a requirements document and the necessary domain knowledge around requirements development. My software architecture course also was and will be applicable, as it helped to understand better and design the format of the system. Choosing between different architecture styles was a lot more straightforward due to this. Due to the nature of the project 4HC3 will be highly applicable as development begins, especially when it comes to usability testing.

A.4.3 Mason Azzopardi

Comment: *This section should not be empty!*

1. The Stakeholder analysis, it went so much smoother than it has in the past because of my proximity to the stakeholders. Also the High level usage scenarios and detailed usage scenarios went very smoothly compared to 3RA3, I think the larger technical involvement of this project made them easier to write out.

2. The main pain points of this submission for me was the hours needed to put in, I had a lot of things going on during reading week and managing my time ended up being very difficult, the solution I had to this was

finding the smallest gaps in my schedule to work, for example I reviewed PR requests while I was watching my roommates cross country meet.

3. I think courses like 2AA4 and 3A04 will be the most helpful for this project, because they focus on the fundamentals of system design and one of the main goals of the project is the inheritence of the project by future software teams, and solid software design principles are needed to do that.

A.4.4 Michael Roberts

Comment: *This section should not be empty!*

1.

2.

3.

4.

A.4.5 Jack Easton

Comment: *This section should not be empty!*

1. I think during the writing of this deliverable the team did a good job of coming together (despite difficulties scheduling and working through reading week) to create a uniform concise document that appropriately represents the design of the system we are creating for the Troitsky team. I think we've done a good job creating requirements that will lead to a polished and producitive end result that will benifit the McMaster Troitsky Club.

2. I think the biggest pain point in this deliverable is just the sheer scale, depth, and dependencies between sections across these two documents. It was very difficult intially to get a handle on what order to approach tasks, what needed to happen first, and when we needed to cricle back to a previously written section and revaluate the thngs we had written. We resolved these early on by mapping out and breaking down the deliverable into 3 internal milestones and set internal deadlines for these to promote timely work and provide a realistic framework for achiveing the work we had to get done.

3. By far the course that has been the most helpful is "SFWRENG 3RA3: Software Requirements and Security Considerations". This course taught me the importance of requirements and how to effectively write and break down problems into necessary, verifiable, and attainable requiements. I think "SFWRENG 3A04: Large System Design" also has been very helpful, and will continue to be helpful in the design and development processes of this project. The course teaches how to design large systems (such as

the Troitsky Toolbox we are developing) efficently and in a manner that will make later development as straightforward as possible.

A.4 Generative AI Use Disclosure

A.4.1 AI Tools Used

- ChatGPT 5.2 (OpenAI)
- Claude Opus 5.5 (Anthropic)

A.4.2 How and Where Used

AI tools were used during this deliverable to assist with the following:

- **Formatting:** help with LaTeX formatting, building, and layout (tables, lists, and section structure) across the document.
- **Linking:** adding and fixing internal references and labels between sections and requirements within the SRS.
- **Review:** reviewing drafted sections for clarity, consistency, and possible errors or omissions.
- **Cross-referencing between documents:** updating and checking references between the SRS and the Hazard Analysis, so that the functional and non-functional requirements cited in the Hazard Analysis FMEA table match the current SRS.

A.4.3 Verification of AI-Generated Content

The team followed the same policy for AI use that the Development Plan sets for AI-generated code: each developer shall thoroughly review all AI-generated content before creating a pull request and not submit the content for a pull request until they completely understand it and are convinced of its validity.

- Each team member reviewed any AI-assisted changes to their own sections before opening a pull request.
- Cross-references between the SRS and the Hazard Analysis were checked by hand against the current version of both documents.
- Pull requests were reviewed by at least one other team member before merging.
- The team is responsible for the accuracy, completeness, consistency, and appropriateness of this document, whether material was written by a person or with help from an AI tool.

A.5 Reflection

Team

1. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?

2. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.

3. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Individual

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.

A.5.1 Team

1. The majority of our requirements came from our clients and their proxies; we are developing a service for a small club full of engineering students in alternate disciplines to ours, we do not have the background to write the requirements needed for this project without asking those more closely involved in the club. Additionally the size of the club makes it very realistic to at least survey the majority of the members and get a strong idea of what is needed before writing out requirements. In particular I think the functional requirements are more closely related to our conversations with our stakeholders, while the more technical requirements within the non-functional requirements we had the technical background to cover the best.

2. Below is a list of knowledge and skills that each teammate will need to learn during the corse ofthis capstone

    - **Mason Azzopardi:** Will need to learn how to work with third party dependencies in order to preform FEA on STEP file in C#.

- **Jack Easton:** Will need to get familiar with C# as he only has a bit of experience working with it in his course work.
- **Michael Roberts:** Will need to get familiar with front end integration using JavaScript and Razor.
- **Pedro Azevedo:** Will need to learn to integrate the project within the day to day of the Troitsky club.

3. Below is how each group member will work on their skills/ knowledge.

    - **Mason Azzopardi:** Will read through documentation or use Gen AI to explain the dependencies to him. Most likely will read documentation first as it is more reliable.
    - **Jack Easton:** Will watch youtube tutorials or go through a free online C# course, most likely will go through youtube tutorials as it fits his learning style more.
    - **Micheal Roberts:** Will use his C# experience to practice with Razor or work with Pedro who has experience with Razor. Most likely the first due to Pedros availability.
    - **Pedro Azevedo:** Pedro will work on the integration of project within the club through interactions with peers and conversations with execs. He will prioritize his interactions with peers because their adoption will drive the executives adoption.

A.5.2 Pedro Azevedo

1. In writing this deliverable, the utilization of GitHub features, such as kanbans, issues, comments, PRs, milestones, tags and even kanban custom views, facilitated development and allocation of work among the team. As the SRS is such a large document, utilizing these features allowed confidence in limited conflicts appearing. The utilization of a LaTeX template with component text documents that are loaded in allowed branches to isolate their features to single documents.

2. The primary pain points came from determining what is expected within this deliverable and working with that in our project domain. As many of the examples given for the SRS are varied, there was a lack of clarity in the necessary depth for certain sections. It was also difficult working around the hyperlinking and interconnection within the document, as updates to sections would have to be reflected in several components.

3. The primary course that has helped so far is 3RA3, as that covered the development of a requirements document and the necessary domain knowledge around requirements development. My software architecture course also was and will be applicable, as it helped to understand better and design the format of the system. Choosing between different architecture styles was a lot more straightforward due to this. Due to the nature of the

project 4HC3 will be highly applicable as development begins, especially when it comes to usability testing.

A.5.3 Mason Azzopardi

Comment: *This section should not be empty!*

1. The Stakeholder analysis, it went so much smoother than it has in the past because of my proximity to the stakeholders. Also the High level usage scenarios and detailed usage scenarios went very smoothly compared to 3RA3, I think the larger technical involvement of this project made them easier to write out.

2. The main pain points of this submission for me was the hours needed to put in, I had a lot of things going on during reading week and managing my time ended up being very difficult, the solution I had to this was finding the smallest gaps in my schedule to work, for example I reviewed PR requests while I was watching my roommates cross country meet.

3. I think courses like 2AA4 and 3A04 will be the most helpful for this project, because they focus on the fundamentals of system design and one of the main goals of the project is the inheritence of the project by future software teams, and solid software design principles are needed to do that.

A.5.4 Michael Roberts

Comment: *This section should not be empty!*

1.

2.

3.

4.

A.5.5 Jack Easton

Comment: *This section should not be empty!*

1. I think during the writing of this deliverable the team did a good job of coming together (despite difficulties scheduling and working through reading week) to create a uniform concise document that appropriately represents the design of the system we are creating for the Troitsky team. I think we've done a good job creating requirements that will lead to a polished and producitive end result that will benifit the McMaster Troitsky Club.

2. I think the biggest pain point in this deliverable is just the sheer scale, depth, and dependencies between sections across these two documents. It was very difficult intially to get a handle on what order to approach tasks, what needed to happen first, and when we needed to cricle back to a previously written section and revaluate the thngs we had written. We resolved these early on by mapping out and breaking down the deliverable into 3 internal milestones and set internal deadlines for these to promote timely work and provide a realistic framework for achiveing the work we had to get done.

3. By far the course that has been the most helpful is "SFWRENG 3RA3: Software Requirements and Security Considerations". This course taught me the importance of requirements and how to effectively write and break down problems into necessary, verifiable, and attainable requiements. I think "SFWRENG 3A04: Large System Design" also has been very helpful, and will continue to be helpful in the design and development processes of this project. The course teaches how to design large systems (such as the Troitsky Toolbox we are developing) efficently and in a manner that will make later development as straightforward as possible.